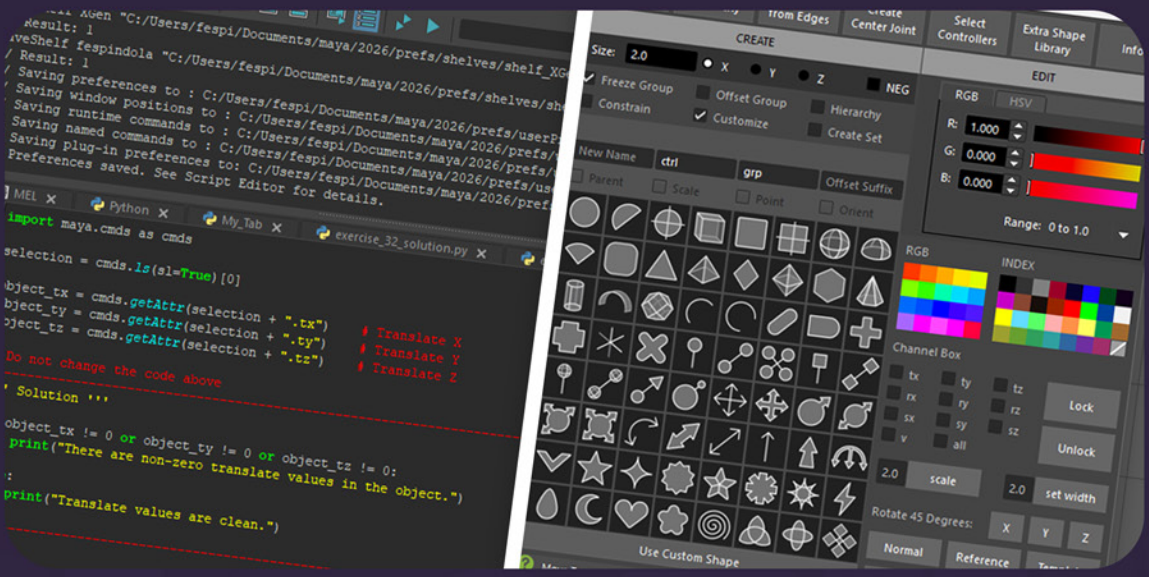
Learn. Create. Master.

First Edition.

The Technical Artist's Guide to Python in Maya.

A visual introduction to tools and automation.

Maya Tools Python



Zeynep Fisher.


Jettelly

The Technical Artist's Guide to Python in Maya.

A visual introduction to tools and automation.

Zeynep Fisher

#Maya #Tools #Python

First Edition 2026

The Technical Artist's Guide to Python in Maya, version 0.0.2.
Published by Jettelly Inc. ® All rights reserved. jettelly.com
77 Bloor St. West, Suite #663, Toronto ON M5S 1M2, Canada.

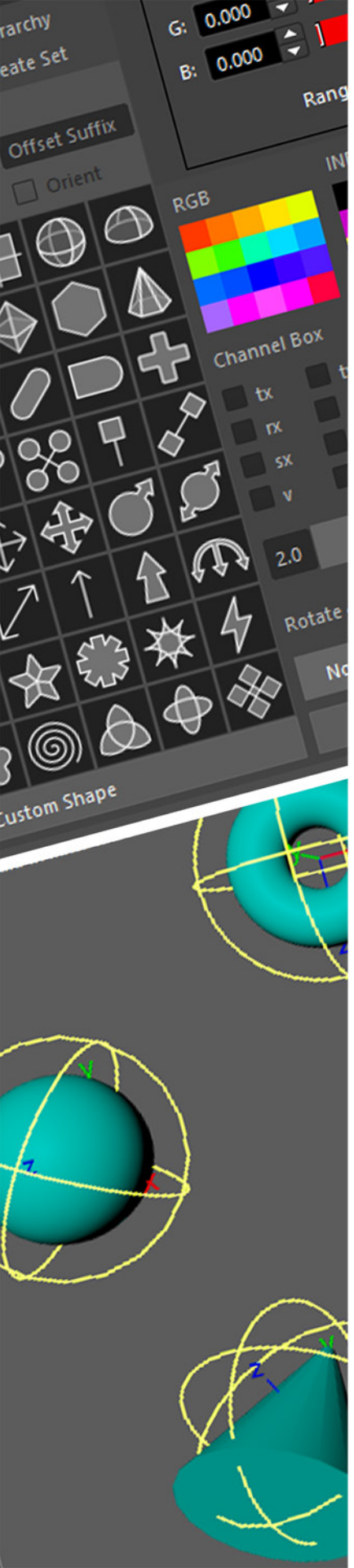
Credits.

Author.
Zeynep Fisher

Design.
Pablo Yeber

Technical Review.
Fabrizio Espíndola

Head Editor.
Ewan Lee



Content.

Introduction. **5**

Studying Tips. 5

Who this book is for. 6

About the Author. 6

Errata. 7

Piracy. 7

1. Getting Started. **8**

1.1 Maya's Programming Options. 9

1.2 Maya Commands vs. Maya API. 13

1.3 Maya's Python Libraries. 14

1.4 Python Version. 16

Summary. 16

2. Course Setup & Maya Script Editor. **17**

2.1 Course Resources & Exercise Files. 18

2.2 Script Editor Window 20

2.3 Script Editor Tools. 23

2.4 Command Line. 28

Summary. 30

3. Python Concepts. **31**

3.1 Printing 33

3.2 Syntax 37

3.3 Data Types 39

3.4 Variables 45

Studying Tips.

I assume many readers of this book will be artists who are completely new to the world of coding. If that is you, you might think that coding is daunting, and that its text-based nature is far too complex for an artist's visually oriented brain. This was exactly my impression when I first started to code. I had many frustrations about whether I would ever truly understand it. Now I can confidently say that with the right approach, every artist can learn to code. Of course, it is not an easy journey, and at times it can be difficult to maintain motivation. This is why, based on my experience, I would like to share some studying tips that can make your learning more efficient and successful.

Exercise your coding muscles.

Coding will feel confusing at first because it is just text, which, sadly, is something we artists tend not to enjoy. But I promise it gets easier, and the key is to do small coding exercises **consistently**. Dr. Angela Yu shares a wonderful analogy that explains a solid approach to learning to code, which I found motivational in my learning journey. To paraphrase, you simply cannot build muscle by going to the gym for only a few days and exhausting yourself with every kind of training at once. Instead, you would probably injure yourself and conclude that you do not like exercising. Similarly, you cannot build muscle if you only train a few times a month. Coding is also like training, but with our brains. To see real progress, you need to practice frequently and consistently, gradually increasing the difficulty rather than jumping into complex programs within days of starting. This is especially important if coding seems like an alien subject to you. By gradually grasping its logic and solving simple problems, you will eventually be able to make your own tools.

In short, try to practice coding for at least twenty minutes each day. This book provides exercises for you to work on. Keep in mind that practice does not mean creating a new project every day; on some days, you can revisit completed exercises and try them in different ways or add your own elements. The key is to stay consistent and turn it into a habit.

Understand the logic, do not memorize.

When you are learning to code, it is not necessary to memorize every command. What matters more is understanding how the code works and starting to think like a programmer. Once you grasp the underlying logic, you can always look up the syntax or specific functions whenever you need them. As you progress, there will be times when you cannot recall the exact method for a given task, and that is perfectly normal. Do not feel bad about it. Even

experienced developers look things up all the time. The important thing is knowing what is possible and having enough understanding to search for the right solution.

It is not just about knowing what to do, but understanding why it works. Keep asking questions, keep observing, and keep exploring why a piece of code behaves the way it does. That is how real understanding grows.

Remember your artistic strengths.
As artists, you already have skills that translate surprisingly well into coding. You are trained to visualize abstract concepts, to recognize patterns, and to break complex ideas into clear shapes and steps. This ability to sketch out systems is a major advantage in programming. As we progress through the book, you will see that we create many flowcharts and diagrams to map out code logic and design tools.

Embrace the journey and challenge.
Learning something new can often feel uncomfortable, but do not let these challenges diminish your motivation. Sometimes a small problem can take a long time to resolve, but as long as you are learning from the experience, the pace of your progress does not matter.

Who this book is for.

This book is designed for Autodesk Maya 3D artists, riggers, animators, and all visual learners who have no prior coding experience and who want to learn how to automate and streamline their creative workflows with the power of Python scripting.

About the Author.

Zeynep is a Game Artist & Rigger experienced in mobile games. Her passion for improving 3D creative workflows led her to explore coding and develop custom tools that make art production more efficient and enjoyable. After starting her career in Istanbul, she moved to London a few years ago, where she currently teaches online rigging classes while specialising in Technical Art through her master's degree.

Errata.

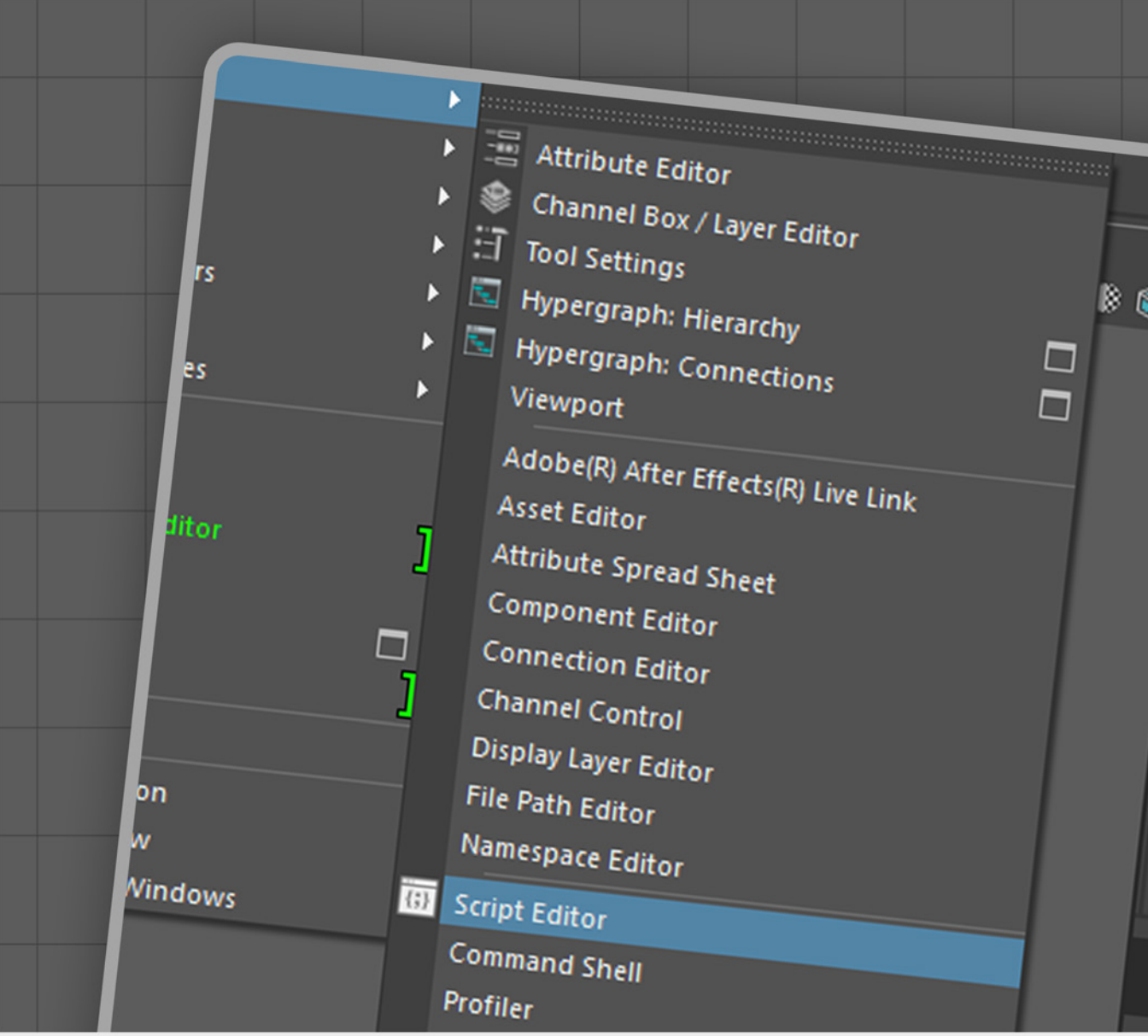
While writing this book, we have taken precautions to ensure the accuracy of its content. Nevertheless, you must remember that we are human beings, and it is highly possible that some points may not be well-explained or there may be errors in spelling or grammar.

If you come across a conceptual error, a code mistake, or any other issue, we would appreciate it if you could send a message to contact@jettelly.com with the subject "AGPSM Errata." By doing so, you will be helping other readers reduce their frustration and contribute to improving future editions of this book.

Piracy.

Before copying, reproducing, or distributing this material without our consent, it is important to remember that Jettelly Inc. is an independent and self-funded studio. Any illegal practices could negatively impact the integrity of our work.

This book is protected by copyright, and we will take the protection of our licenses very seriously. If you come across this on a platform other than www.jettelly.com or discover an illegal copy, we sincerely appreciate it if you contact us via email at contact@jettelly.com(and attach the link if possible), so that we can seek a solution. We greatly appreciate your cooperation and support. All rights reserved.



Chapter 1

Getting Started.

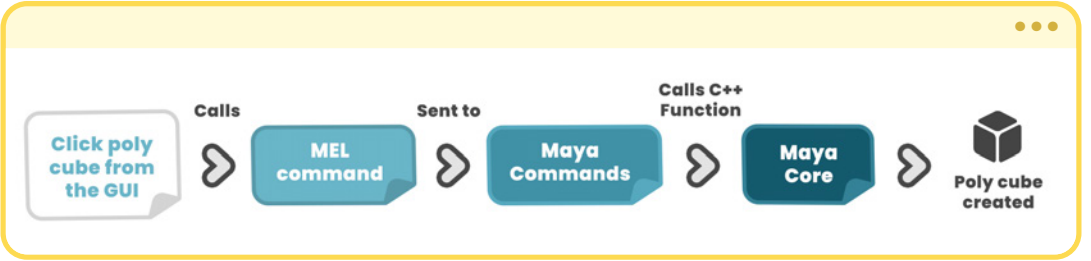
I remember while working as a Game Artist, there were many tasks I wished I could automate or simplify in Autodesk Maya, but I was afraid I would never understand coding. When I finally decided to start coding, finding the right course and approach was a challenge in itself, even before learning to code. I tried many courses, methods, and even children's books! While each of these resources contained valuable information, I was only able to grasp the logic of coding by combining what I learned from them. As a visual learner, I needed a more hands-on and artist-friendly approach that would guide me step by step from Python to Maya scripting concepts.

This book is the result of that journey. Its aim is to provide a clear, practical, and accessible introduction to Python for Maya artists, bringing together the methods, explanations, and workflows that proved most effective in my own learning process as a Game Artist, and guiding you as you begin to build your own tools.

This chapter introduces the fundamental concepts required to begin scripting in Python and Maya. It outlines who this book is intended for, the learning outcomes, and some studying tips to support your learning journey. It also provides an overview of Maya's programming structure, including its Python libraries. Together, these topics establish the theoretical foundation needed for developing tools in Python for Maya.

1.1 Maya's Programming Options.

Before we dive into Python, it is helpful to understand Maya's programming architecture and how it works. There are several ways we can interact with Maya, and as artists, we typically use the most basic method, which is Maya's graphical user interface (GUI). Without coding, we utilize shelf buttons, menu items, and keyboard shortcuts to do everything inside Maya. Whenever we click or select something, Maya is busy behind the scenes: Maya first calls the relevant MEL command, which is then passed to Maya Commands. This, in turn, triggers the corresponding C++ function that Maya Core can understand and execute. See the example flow in Figure 1.1.a.



(1.1.a Internal flow to execute a function)

Note

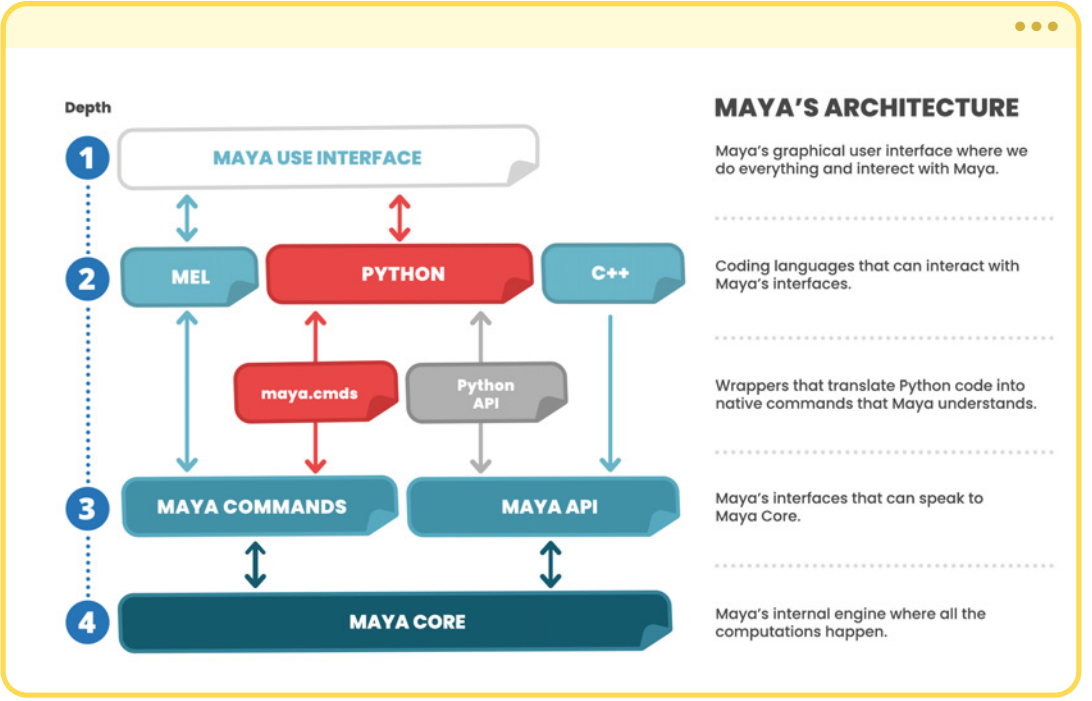
Maya Core refers to the internal engine of Autodesk Maya, the underlying C++ system that performs the actual calculations, data processing, and execution of the operations inside the software.

Alternatively, Autodesk provides several other ways to interact with Maya Core through coding, as detailed in Maya's Architecture diagram in Figure 1.1.b. As you can see, there are three coding languages we can use to extend Maya. Another key point in Maya's Architecture is that there are different layers before the code reaches Maya Core. These layers are created to give users different levels of control and accessibility. The code within Maya Core is highly complex, so it is typical for software to wrap such a core in protective layers. This lets users interact with the Core without risking damage to the underlying engine. Additionally, layers provide us with scripting options at different complexity levels, and thanks to that, artists can start their coding journey in Maya by using simpler approaches without needing to deal with complex topics.

Note

Maya's documentation lists C++ together with .NET as a way to extend Maya. Since .NET is a developer platform and not a programming language, it is not included in Figure 1.1.b to keep the diagram simple for beginners. For more details, see the Developer Help section of the official Autodesk Maya documentation.

In this book, we will learn the Python language and the `maya.cmds` layer, which is a great way to start coding. Let's examine the different programming options in detail.



(1.1.b Maya's architecture)

Maya commands.

In simple terms, Maya Commands contain built-in instructions that tell Maya what to do. These instructions can be called using either MEL or Python. Everything we can do through Maya's GUI can be automated and extended with Maya Commands. Because it is based on predefined commands, it offers ease of use and rapid scripting rather than flexibility.

Maya API.

The Maya API is a flexible and powerful way to extend Maya. Instead of calling simple, ready-made commands, you work directly with objects and nodes. This means the Maya API allows you to extend Maya beyond its built-in capabilities and add entirely new features.

Originally, the Maya API could only be used with C++. While this is still a powerful method, it can slow down the process of writing and testing code, in other words "prototyping". The reason is that C++ code must be compiled before it can be tested. Compilation is the process of converting human-readable code into machine code (binary), which the computer can understand and execute. Also, the compiled code must be loaded via Maya's plug-in manager every time, which can become tedious. With the addition of the Python API to Autodesk Maya, these steps can be bypassed, significantly speeding up the coding

process. This is one of the reasons why many people prefer Python for developing Maya tools.

MEL (Maya Embedded Language).

MEL was created to be used with Maya, and in fact, Maya's GUI is primarily written in MEL. Before Python was introduced, MEL was the only way to access Maya Commands. While it remains Maya's native scripting language, it has certain limitations. First, it has a smaller user base than Python, which can make it harder to find information and troubleshoot. Second, unlike Python, MEL does not support object-oriented programming, an approach that organizes reusable code around objects. We will discuss this topic further in later volumes.

Python.

Python is an easy-to-read programming language first created in 1991 and added to Maya in version 8.5. It was initially introduced as an alternative to MEL to access Maya Commands. Later, the Python API was created as an alternative to C++, making Python the only language that can communicate with both Maya Commands and the Maya API.

Python goes through an interpretation layer that translates it into C++ before it can communicate with the Maya Core. Although this extra step makes its execution slower than C++, Python remains the industry standard for developing pipeline tools due to its simple syntax and rapid development. For Technical Artists, Python is fast enough and capable of building tools complex enough to handle most tasks in Maya. It also has extensive libraries, which are collections of prewritten code that let you perform actions without writing them yourself, along with built-in functions that make coding faster and easier.

C++.

C++ is the language Maya was created in, and it provides the maximum level of control and flexibility for adding new features to Maya. It also runs faster because it can communicate directly with the Maya Core, unlike Python, which must go through the interpretation layer. However, it is essential to point out that C++ has a steeper learning curve, and its compilation requirements can also make prototyping slower, as mentioned in the Maya API section. For these reasons, C++ is usually reserved for performance-heavy tasks or complex pipelines, where large amounts of data, files or nodes need to be processed, while Python is preferred for most technical art tools due to its simplicity and quick development process.

1.2 Maya Commands vs. Maya API.

As we will focus on Python and the `maya.cmds` module in this book, our code will run through Maya Commands. However, it is beneficial to understand when to use Maya Commands and when to use the Maya API.

According to Autodesk's official documentation, Maya Commands are used for scripts, while the Maya API is intended for plug-ins. But how can we determine whether a given tool needs a script or a plug-in? The following list outlines the primary differences that can help clarify this choice:

Scripts.
These are typically used for quick, artist-focused tools.

- Automate repetitive tasks or streamline art workflows that would normally take a long time to complete manually.
- Quickly modify attributes or geometry.
- Create customized actions, windows, or hotkeys.
- Run from Maya's Script Editor.
- Remain easy to write, test, and share.

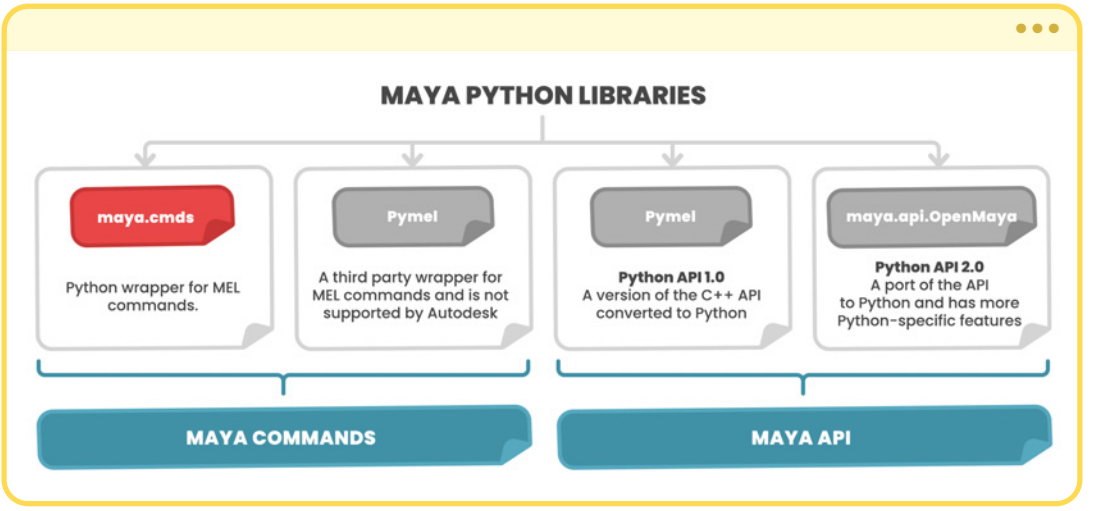
Plug-ins.
These are for creating custom nodes, new object types, and completely new features to extend Maya at a deeper level.

- Provide greater flexibility.
- Support production-scale pipeline tools capable of handling performance-heavy tasks.
- Load through the Plug-in Manager.
- Require more setup and deeper technical knowledge to develop and maintain.

When you begin creating your tool, it is always good to ask yourself the following question: Does the tool I want to write require a Maya command that does not exist? If the answer is yes, you can create such a command by writing a plug-in using the **Maya API**. If the answer is no, scripting through **Maya Commands** will be sufficient.

1.3 Maya's Python Libraries.

Maya comes packaged with four Python libraries. As you start developing your Python tools for Maya, you will likely encounter names other than `maya.cmds`, as there are additional Python libraries available. Figure 1.3.a provides a summary of each library.



(1.3.a Maya Python libraries)

We will discuss object-oriented programming in detail in the upcoming volumes, as this volume focuses on procedural programming. However, it is important to briefly review these concepts to better understand the differences among the libraries listed above.

Procedural programming means writing code as a set of step-by-step instructions, like a recipe. For example, you tell the computer: first create this, then move it, then scale it.

Object-oriented programming, a.k.a. **OOP**, means working with "objects" that contain both data and actions. Instead of only giving instructions, you work with things that have their own **attributes (data)** and **methods (actions)**. For example, a car object can have attributes like brand, colour and speed, and methods like drive, turn and stop. Similarly, in Maya, models, joints or anything we create can be treated as an object in coding.

When it comes to coding with **Maya Commands**, you have two options: `maya.cmds` and `pymel`. `maya.cmds` is procedural and command-based in style. It tells Maya which command to run, step by step. The important point is that these commands do not return objects with their own attributes and methods; instead, you get results like the name of an object

or some data that is not attached to anything. For example, when we create a cube, Maya returns its name, such as `"pCube1"`. To move or rotate it, we use separate commands and pass the cube's name to them. In other words, the cube does not know how to move itself. In an object-oriented approach, the cube would have its own move or rotate behaviour, and we could trigger those actions directly. However, even though `maya.cmds` is procedural in nature, you can still write object-oriented programs by combining it with Python, which itself is fully object-oriented. In other words, you can use `maya.cmds` inside a program that is structured as OOP. Because `maya.cmds` is Autodesk's official wrapper and is well-documented, it is a great way to start developing tools in Maya, which is why we are focusing on it in this book.

`pymel` is very similar to `maya.cmds`, but the key difference is that it is **object-oriented** in style. It automatically wraps Maya nodes into Python objects, allowing you to interact with them directly. It is slightly slower and less commonly used in production pipelines, but once you learn object-oriented programming in later volumes, it can be worthwhile to explore.

To help visualize the difference between the **procedural style** of `maya.cmds` and the object-oriented style of `pymel`, a simplified code example would look like this:

Procedural.

```
move(cube)
```

Command first: the command comes first, then receives the cube as the object to act on.

Object-oriented.

```
cube.move()
```

Object first: the cube comes first, then performs an action it already knows how to do.

When coding with the **Maya API**, there are two main libraries we can use: `maya.OpenMaya` and `maya.api.OpenMaya`. Both are Python wrappers for Maya's core C++ API, allowing users

to perform advanced operations. These operations often require a deeper understanding of Maya's internal data model, and the two libraries differ in version and style.

- `maya.OpenMaya` is the original, older version of the API wrapper. It is low-level and verbose, meaning that even simple tasks often require extra steps. You are most likely to encounter it in older plug-ins or legacy pipelines.
- `maya.api.OpenMaya` is the newer version, designed to be more convenient and suited to Python (or "Pythonic"). It is faster and more efficient for plug-ins that manipulate many nodes or large amounts of geometry, and is generally considered the preferred API coding library today.

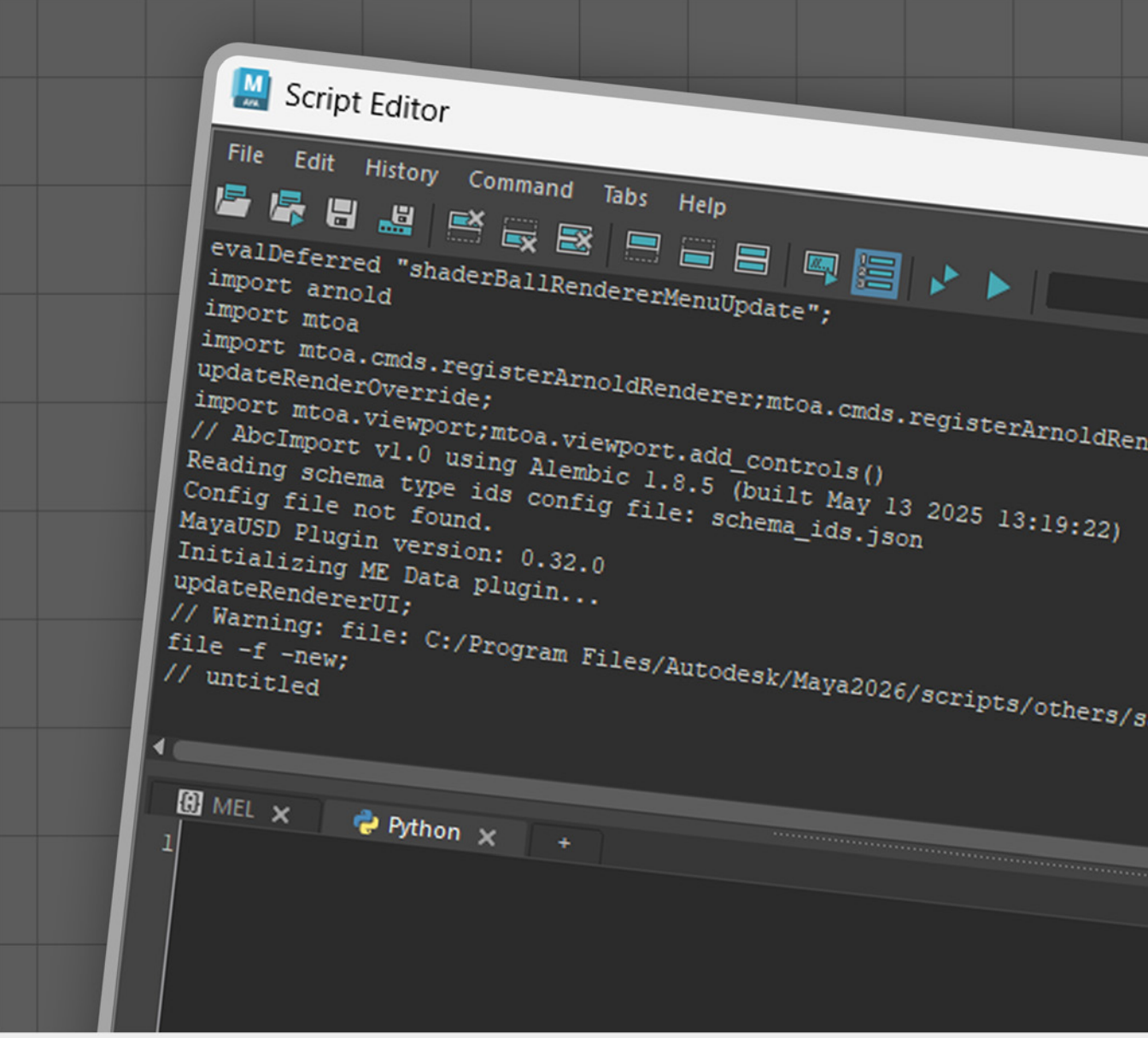
In short, `maya.cmds` and `pymel` are for accessing **Maya Commands**, while `maya.OpenMaya` (Python API 1.0) and `maya.api.OpenMaya` (Python API 2.0) are for accessing the Maya API.

1.4 Python Version.

Maya 2023 and above use Python 3 exclusively. In this book, we will focus on the latest version, **Maya 2026**, and learn **Python 3**, which is the preferred and modern standard. Python 3 addresses some of the design issues found in Python 2 and provides greater readability. Because of the changes to its syntax and behaviour, Python 2 code generally does not run in Python 3, and vice versa. Therefore, always verify which Python version your edition of Maya uses.

Summary.

In this chapter, we laid out this book's aim and structure, along with study and motivation tips to help you on your learning journey. We learned about `maya.cmds`, the main subject of this book, and where it sits in relation to other Maya programming options.



Chapter 2

Course Setup & Maya Script Editor.

As this book is designed specifically for Python coding in Maya, we will use Maya's features to write and execute our code. This chapter first shows you how to set up the course files for this book, then it guides you on how to open and use Maya's Script Editor and provides detailed instructions on its UI elements and tools. By the end of this chapter, you will become familiar with the Script Editor setup, so you can start writing code and work through the examples and exercises in the next chapter, where we begin learning Python.

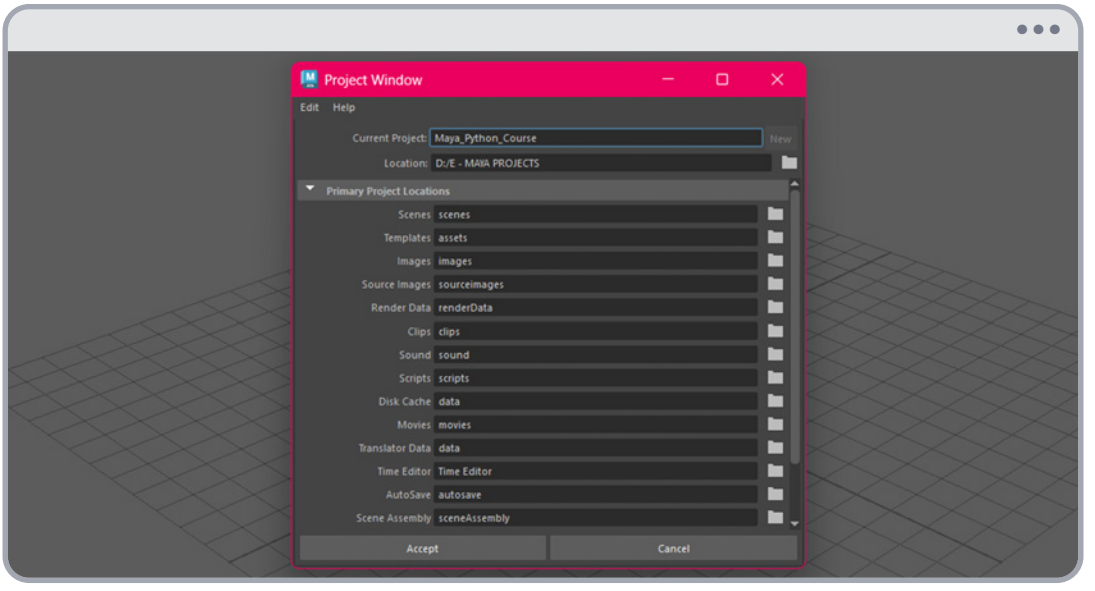
Maya's Script Editor is sufficient for beginner-level coding; however, in the next volumes of this book, we will switch to IDEs (Integrated Development Environments), as they make a real difference to the coding experience. If you already have an IDE and feel comfortable using it, you are welcome to do the coding exercises in this book within that IDE. Just remember that using Maya Commands in an IDE requires an extra setup process that will probably be explained in a different book.

For those who are wondering what IDEs are, they are software programs that allow coders to write, test, and debug code in one place. As they offer auto-completion, syntax highlighting, and point out potential errors before you even execute your code, they are highly efficient tools for developers. Maya's Script Editor is admittedly quite primitive compared to IDEs. Even so, setting up and working with an external IDE can introduce additional complexity for beginners and may distract from the core goal of learning to code. It is also valuable to learn Maya's scripting quirks and unique behaviours. For these reasons, this volume focuses on coding entirely within Maya.

2.1 Course Resources & Exercise Files.

Let's start this section by creating a new project within Maya:

1. Go to File and select Project Window.
2. Press the "New" button to create a new project.
3. Name this project "Maya_Python_Course." You can use another name if you want.
4. Select a preferred location and accept the project.



(2.1.a Project configuration)

Note

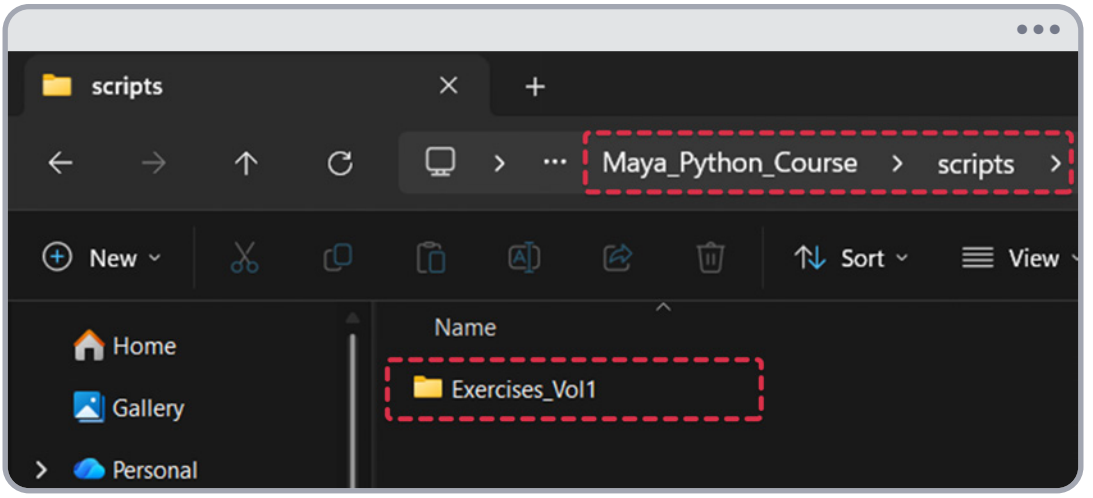
If you want this project to open every time Maya is running, go to File and select Set Project. Then, select the current project we are working with.

Before you start coding, make sure to download the resources for this book directly from the "My Courses" section of your Jettelly account. As we move through the coding topics, you will find short exercises in the project folder that help you practise what you have learned. Each exercise has its own sub-folder inside the main exercise folder. Each sub-folder includes:

- PDF file with instructions and the solution.
- Starter code file (if required).
- Solution code file.
- Maya scene file (if required).

Please follow the instructions below to set up the exercise folder on your computer:

1. Download the entire Files folder from your account.
2. The Folder may appear as a .zip folder.
3. Inside, you will find a folder named **Exercises_Vol1**. Copy or move this folder into your Maya version's scripts folder.



(2.1.b Exercises in the script folder)

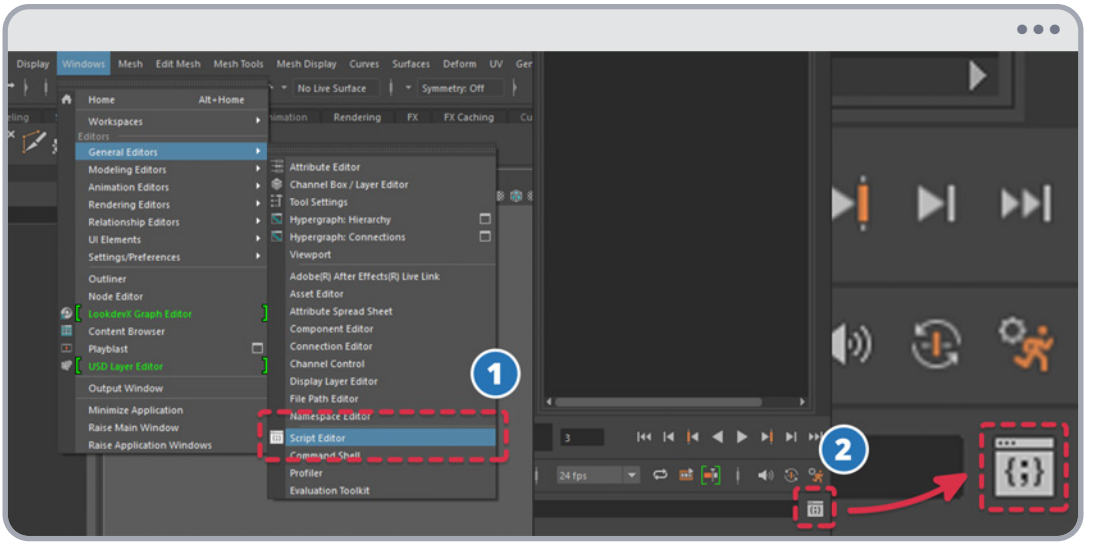
The first ten exercises (1–10) are foundational. They reinforce basic topics, require only a few lines of code, and ask you to write that code by following the instructions. From exercise 11 onward, the exercises generally provide an incomplete code sample for you to finish to complete the challenge. These later exercises are more demanding, designed to help you combine multiple concepts and apply them in more practical scenarios.

You do not need to worry about them now. The next chapter, where the exercises begin, includes notes on how to open and complete them. There will also be optional quizzes linked in the book if you would like to further practise what you have learned.

2.2 Script Editor Window

Let's look at the elements essential for scripting in Maya. Because we will write and run all of our code in the Script Editor, it is worth getting familiar with how it works. To open the Script Editor, you can use either of the following options:

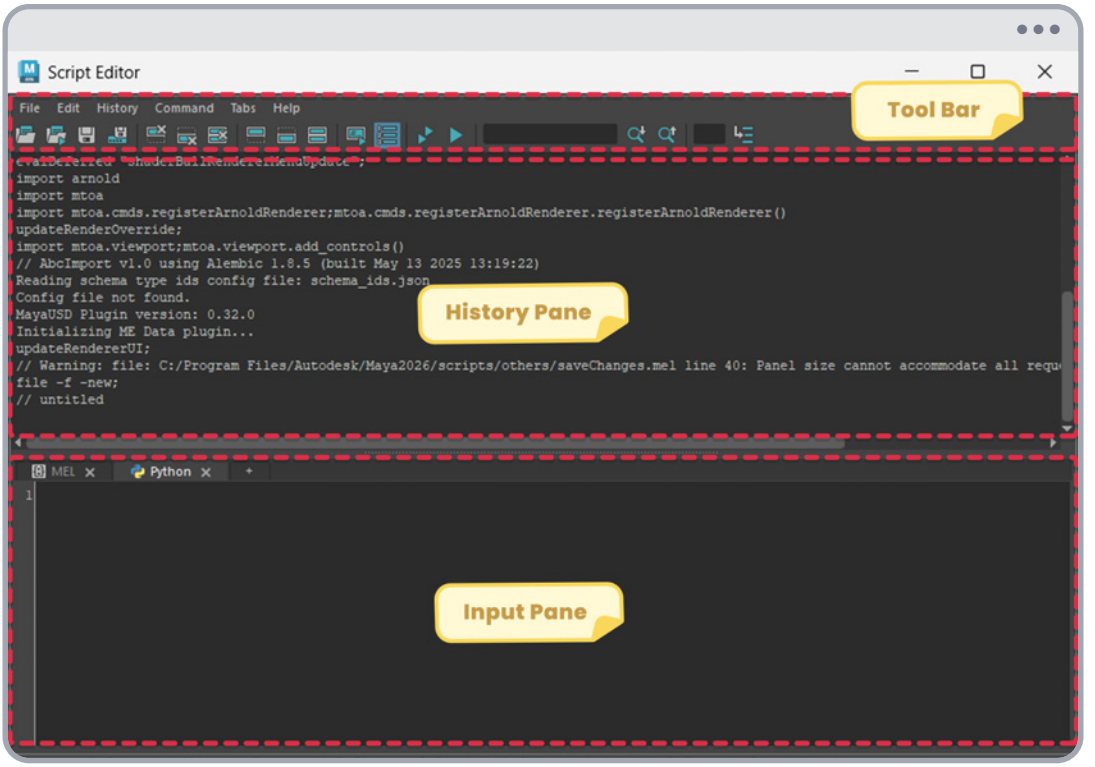
1. Find it through **Windows > General Editors > Script Editor**.
2. Or simply click on the little **Script Editor** icon located at the bottom right corner of Maya, as shown in Figure 2.2.a.



(2.2.a Script Editor location)

Once you open the Script Editor, you will see the window in Figure 2.2.b. It consists of three main sections:

1. **Input Pane** is where you write the code.
2. **History Pane** shows the executed commands and their results. This applies to the code we write as well as to any manual operations performed in Maya. For example, creating a simple poly cube is detailed as a MEL command in the History Pane.
3. **Toolbar** provides icon buttons and menu options for common editing, execution, and management tasks.

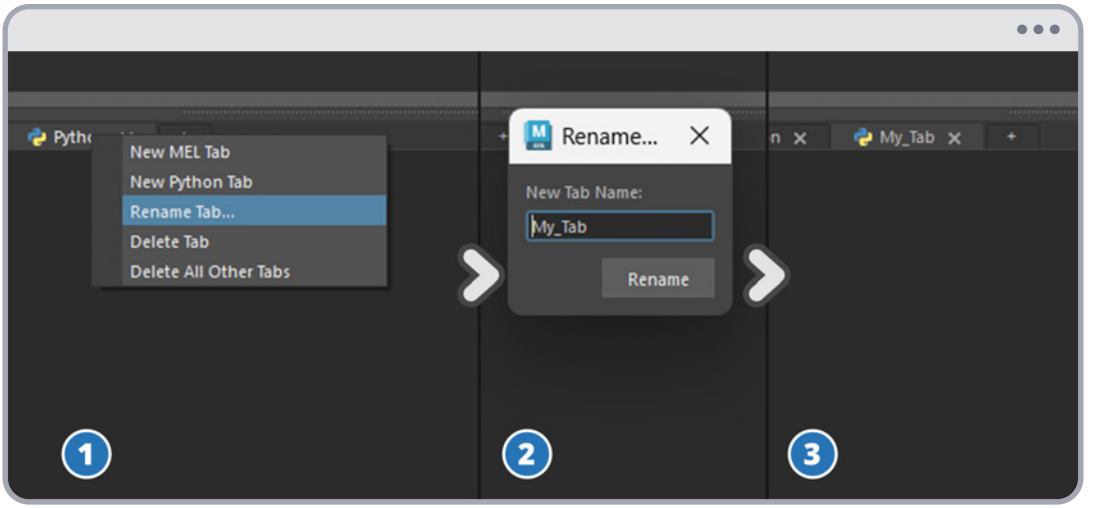


(2.2.b Script Editor window)

If you look closely at the Input Pane in Figure 2.2.b, you will notice two tabs: **MEL** and **Python**. To start coding, we first need to make sure we are writing the code in a Python tab. You can also create new tabs. For example, for each coding exercise in this book, it is better to work in a separate tab.

To create a new tab, simply click on the plus icon next to the tabs. It will ask you whether you want MEL or Python. You can do the same by going to the Tabs menu and selecting New Tab, or right clicking an existing tab and selecting New Python Tab.

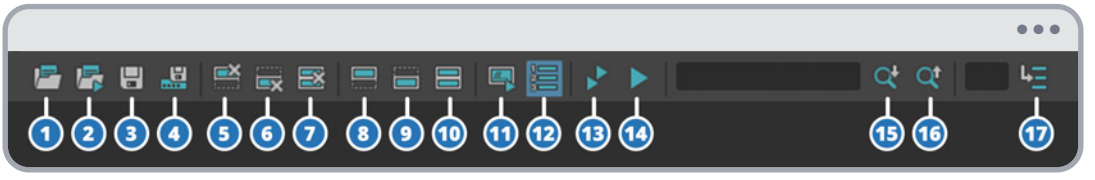
To help you remember what you are working on, you can give your tabs short names. Right-click the tab and select **Rename Tab**, as shown in Figure 2.2.c. Then, enter a new name in the dialogue box that appears. Your tab will display the name you chose, but it will always have the Python icon (a blue-and-yellow snake symbol) to the left of the name to indicate that it is a Python tab.



(2.2.c New Python Tab)

2.3 Script Editor Tools.

Let's review the elements in the toolbar. The icon buttons below are the ones you will use most frequently. While the dropdown menu options mainly provide the same functionality, they also include some additional features. We will focus on the important ones.



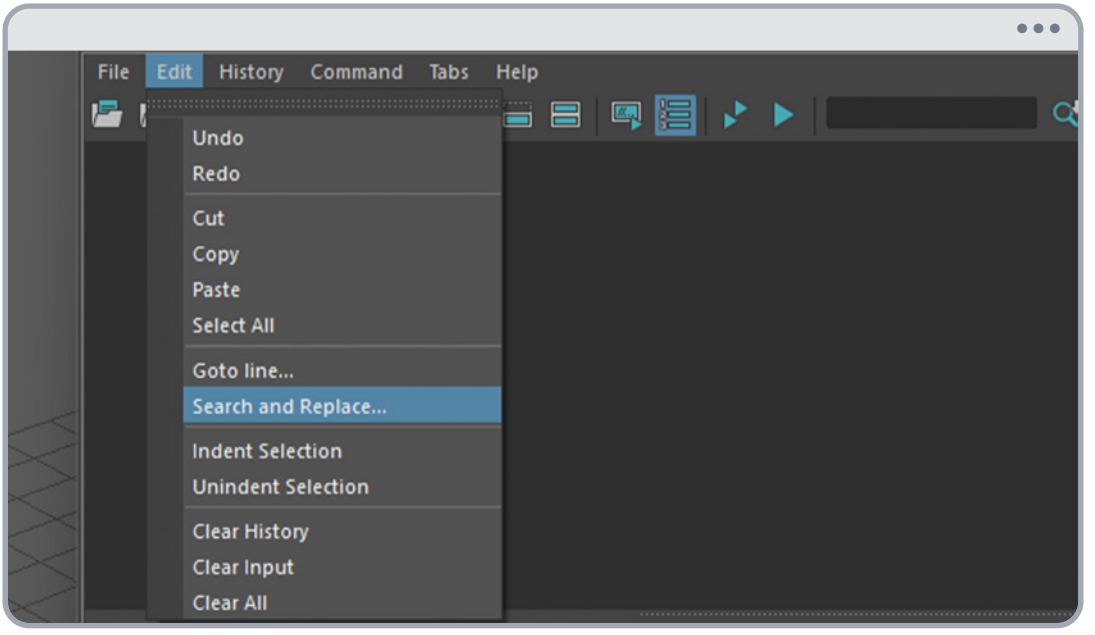
(2.3.a Tool bar buttons)

What do these buttons do?

1. **Load / Open script**: Opens a code file in the Input Pane.
2. **Source script**: Runs a file you choose from disk.
3. **Save script**: Saves the script from the current tab into a .py file on your computer. Preferably, keep your files in the Scripts Folder, as it is the default location for Maya. Keep saving your files frequently and make versions when necessary. This will be good practice, especially when you are a new learner. In the later volumes, as IDEs will be introduced, version control systems such as GitHub will also be covered.

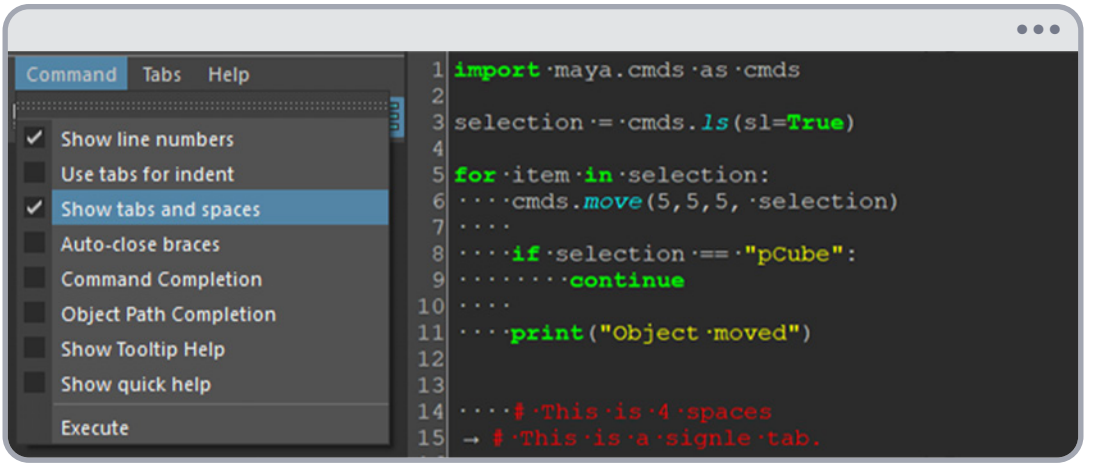
4. **Save script to Shelf**: Creates a shortcut button for the script from the current tab to the current Shelf, which holds icons for commonly used actions and tools. Shelf is usually located above the Outliner and the Viewport. Different tabs have different icons, and since it is customizable, yours might look different.
5. **Clear history**: Clears the History Pane.
6. **Clear Input**: Clears the Input Pane, meaning it deletes your entire code from the current tab, so be careful with this button.
7. **Clear all**: Clears both the History Pane and Input Pane.
8. **Show history**: Maximizes the History Pane.
9. **Show input**: Maximizes the Input Pane.
10. **Show both**: Returns to the standard view by showing both the History and Input Panes.
11. **Echo all commands**: This shows all background commands that you have run in Maya. When it is turned off, Maya usually shows only the main commands you are performing, such as moving an object, while omitting many internal system and background commands. You may want to see a more detailed version of these Maya actions to learn and use for your own code. In those cases, you can turn this option on. I do not recommend keeping it on all the time because it creates a very cluttered History Pane.
12. **Show line numbers**: Turns code line numbers in the Input Pane on and off. I highly recommend keeping this enabled, as when Maya reports an error, it usually includes the line number, making it easy to locate the problem.
13. **Execute all**: Runs all the code from the current tab in the Input Pane.
14. **Execute**: Runs whatever code is currently selected. Use this with caution! If no code is selected, it runs all the code in the Input Pane and then deletes it. If a code selection is made, only that selection will be executed, and the text will remain. There is a little trick for this option: if you select lines but move your cursor outside the Input Pane to click Execute, Maya will still run all the code rather than just the selection. Using the shortcut Ctrl + Enter avoids this issue and ensures only the selected code is executed.
15. **Search down**: Enter a word in the search box located to the left of this button. It searches the Input Pane from top to bottom and highlights each instance of the word (if any). Keep clicking to jump to the next one down.
16. **Search up**: This button does the same thing as Search down, just in reverse (i.e., searching bottom to top).
17. **Go**: Enter the line number in the box to the left of this button and click. It navigates to that line in the Input Pane. This is especially helpful when tackling errors.

Beyond the toolbar buttons, a few menu options are worth knowing because you will use them frequently. For example, **Edit > Search and Replace**. This option replaces the search word with the new word in your code. It can be useful, for example, if you decide to change variable names. Note that IDEs have a specialized and more efficient search and replace function called refactoring, but Maya's functionality can still be helpful for beginner-level code.



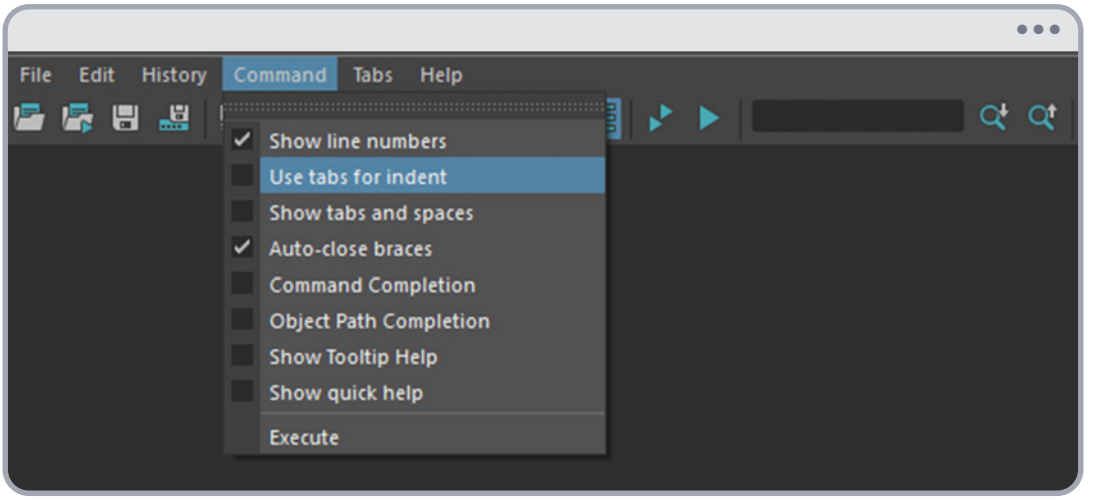
(2.3.b Search and Replace option)

Another useful option is **Command > Show tabs and spaces**. This one displays a dot for each space and an arrow for each tab as shown in Figure 2.3.c. Spaces and tabs, or in technical words, indentation, is an essential part of Python coding, and we will discuss this in the next chapter. Using this option can help you visualize whether you have indented your code blocks correctly. For example, each level of indentation must have four spaces; therefore, Maya will display four dots. As it makes the code a little bit cluttered, you might want to turn it on only occasionally or when you get indentation errors.



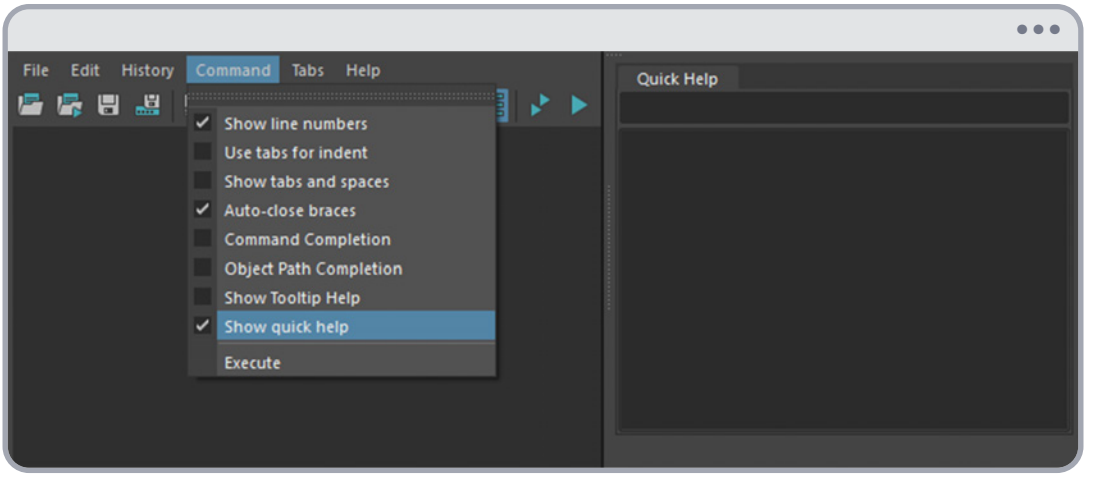
(2.3.c Show tabs and spaces option)

Within the same menu, we can find **Command › Use tabs for indent**. Like most IDEs, Maya's Script Editor inserts four spaces when you press the Tab key. This option lets you toggle between inserting tabs and spaces: when it is unchecked, pressing Tab inserts spaces, which I recommend leaving as the default. We will discuss the official indentation rules, and the ongoing debate between tabs and spaces, in the following sections.



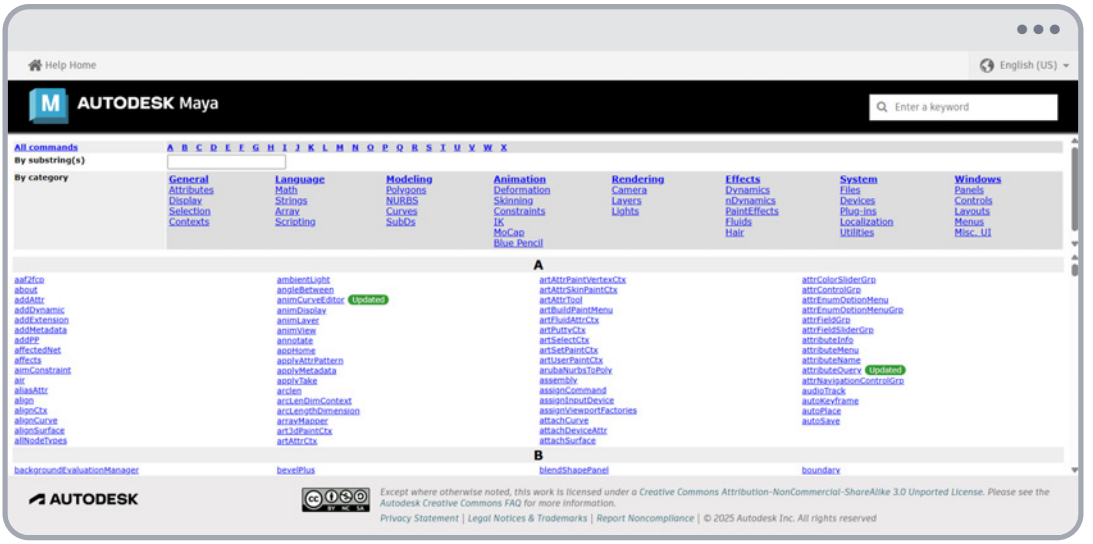
(2.3.d Use tabs for indent option)

The **Command › Show quick Help** option opens a tab next to the **Input Pane**, where you can type the name of a Maya command and it will give you a list of parameters (technically known as "flags") you can use with that command. This will become increasingly useful as you progress in your coding journey.



(2.3.e Show quick help option)

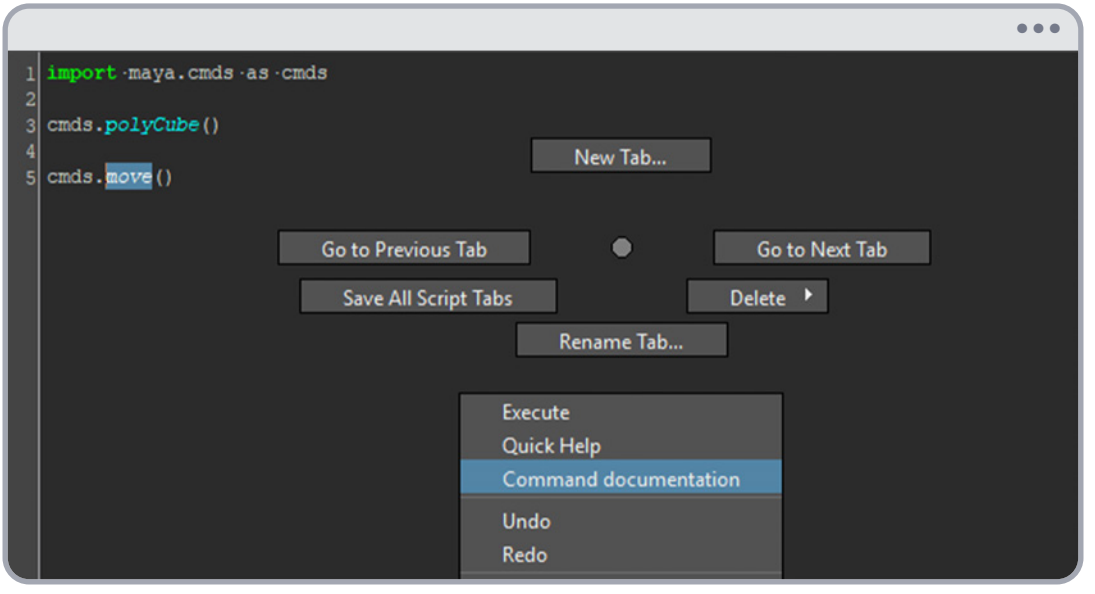
Finally, **Help > Python Command Reference** opens Maya's official Python Command Documentation in your browser, see Figure 2.3.f. Here you can find detailed information about each command, how to use it, and try out the provided example code. You will rely on this reference frequently as you write code, so it is a good idea to always keep it open in your browser.



(2.3.f Autodesk Maya official documentation)

You can also use the marking menu options and shortcuts for various actions. Here are some that might be helpful. To display it, highlight a Maya command, then hold right click on the Input Pane, then select Command documentation. It will open the corresponding

documentation page. This page provides detailed information about the selected command. For example, if you highlight the "move" command and use this option, you will access its Python documentation. Ensure that you highlight the entire word and confirm that the command exists. A helpful tip: Maya highlights its commands in **blue**, so you can easily verify that you typed the command correctly and that it is valid.



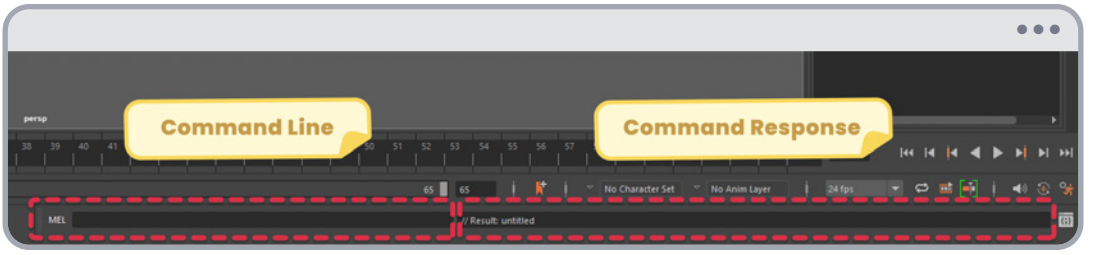
(2.3.g Command documentation)

This panel also includes the Save All Scripts Tabs option. Maya normally saves your open tabs automatically when you exit; however, this feature seems unreliable in newer versions, so saving them manually is a good habit. Saving each tab as a .py file with the Save Script button shown earlier remains the safest way to protect your work, but this option is a convenient backup.

You can also adjust the font size of the Input Pane: hold the Ctrl key and scroll the mouse wheel up or down while your cursor is over the pane.

2.4 Command Line.

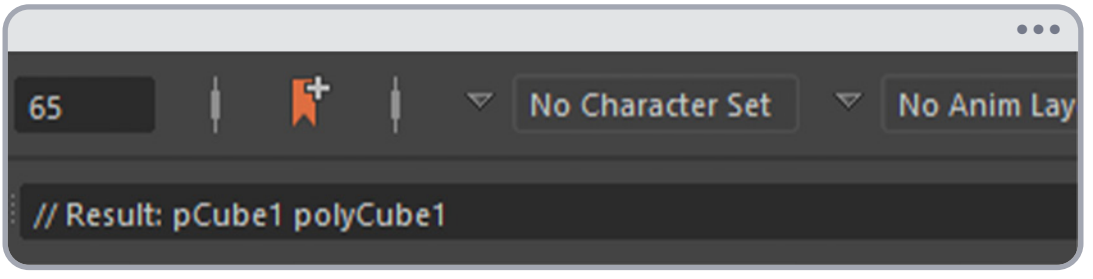
The Command Line is another UI element in Maya that is useful for scripting. It consists of two sections: **Command Line** and **Command Response**, as shown in Figure 2.4.a.



(2.4.a Command Line and Command Response)

Command Line, usually located at the bottom-left, is for running short Python or MEL commands. It accepts only a single line of code, although you can place several statements on that one line. To switch languages, click the name of the code language displayed on the left, which toggles between **MEL** and **Python**.

Command Response, located beside Command Line, is where Maya gives us feedback. You might recognize it from red errors or yellow warnings, but it also shows status messages and code output. For example, when you create a poly cube, it reports the result, as shown in Figure 2.4.b. This feedback is an important part of scripting, since you can create custom messages, errors, and warnings to inform the users of your scripts.



(2.4.b A polyCube has been created in the Scene)

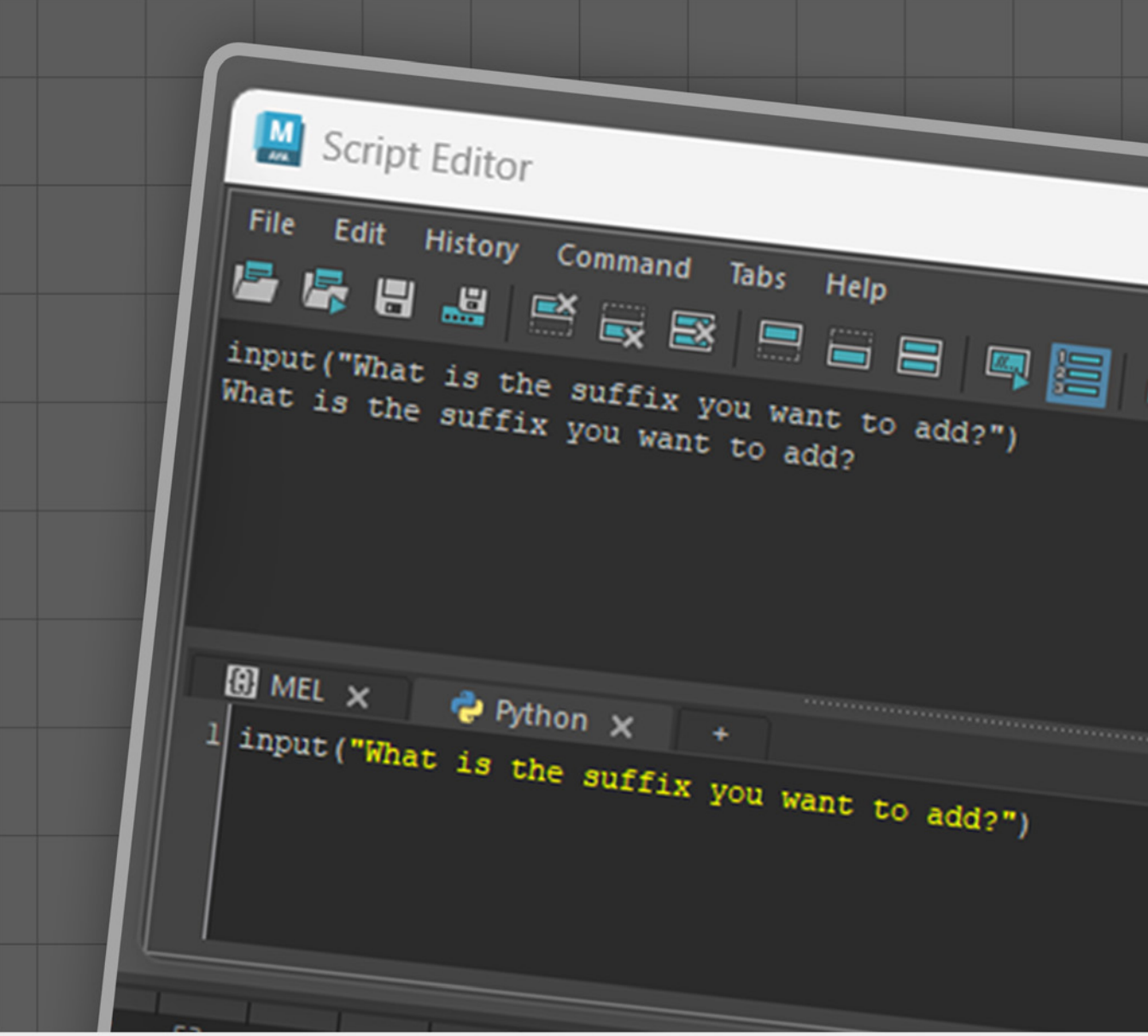
Command Response only displays a short message and reflects the most recent command. To view the full history and output, always use the Script Editor.

Note

If you cannot see the Command Line section in your Maya UI, navigate to **Windows > UI Elements > Command Line**, and make sure it is enabled.

Summary.

In this chapter, we set up the course files and resources, then explored the Script Editor and the role its components play in coding. We worked through each Script Editor tool, explaining what it does, and finished with the Command Line and its Command Response. Together, these establish the fundamental knowledge needed to confidently start writing and experimenting with Python scripts in Maya, a foundation we will build on in the following chapters.



Chapter 3

Python Concepts.

Coding is a broad and specialized field that can take years to fully explore, with countless advanced libraries and techniques to learn. The good news is that the core concepts of programming are far more focused, and when it comes to scripting tools for Maya or other 3D software, the scope narrows even further than what a game or web developer needs to know. This means you can begin creating useful tools without mastering every aspect of programming.

In this chapter, you will learn all the Python fundamentals you need to begin writing your own scripts. We will start with the basics and gradually explore its logic and concepts step by step. Each concept is explained in detail and supported by small code examples that demonstrate how it works in practice, and I encourage you to copy and paste these into your Script Editor to test them out. You will also find short exercises and quizzes designed to reinforce your understanding. Completing these regularly, even in small amounts, is one of the most effective ways to build confidence and develop your coding skills over time.

Of course, as your skills develop, it is valuable to continue learning more advanced techniques that make your code more efficient, practical, and professional. Even so, a strong command of Python's fundamental concepts is enough to build genuinely complex tools, which is why mastering those fundamentals is the focus of this chapter.

Creating things in Maya through scripting involves combining Python with Maya commands. Without a solid understanding of Python basics, it is difficult to use or understand these commands effectively, which is why Python comes first. Throughout this chapter, I have tried to explain concepts with Maya-relevant examples. However, many of them remain theoretical, meaning they simulate actions rather than directly control Maya. This is because we will cover Maya commands in Chapter 4, after first building a strong foundation in Python. That said, I have occasionally scattered a few Maya commands throughout the exercise files to give you a preview of what is coming and to make the learning process more engaging. Learning Python takes time and practice, but once you complete this chapter, everything that follows will feel much more intuitive, and your progress will be significantly faster!

3.1 Printing

Python has built-in functions that, in simple words, tell the computer to perform certain operations. For example, the `abs()` function allows you to calculate the absolute value of a number.

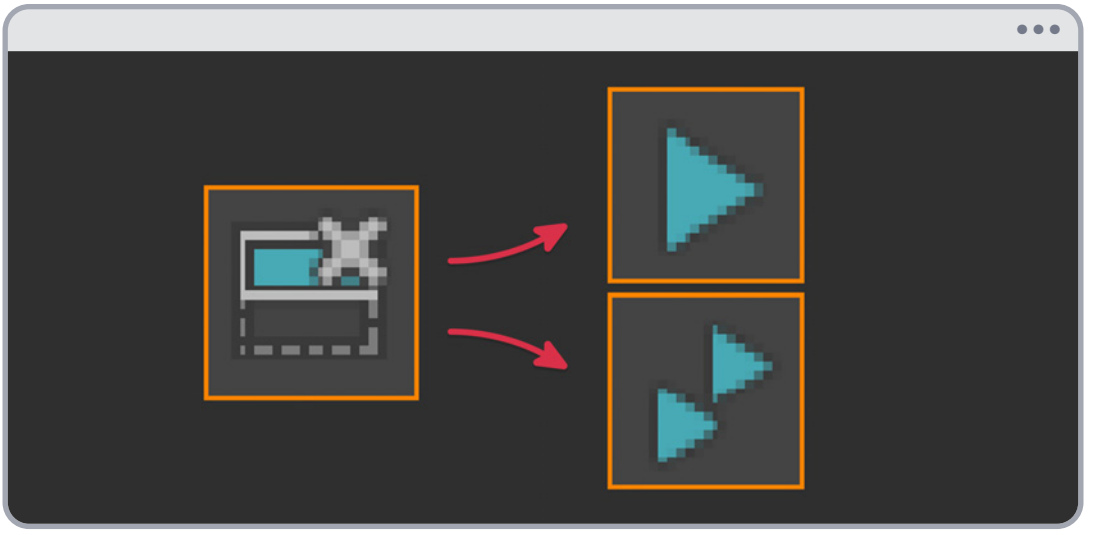
> **Note**
>
> You can find the complete list on Python's documentation page in the following link:
> https://docs.python.org/3/library/functions.html

Among the functions, you can find a very useful one, the `print()` function, also known as the print statement, which is the most basic function in Python, and it makes the computer write a message that will appear in the **History Pane** in the Script Editor, or in the **Console** if you are using an IDE. Let's write our first piece of code.

Go to the **Input Pane** and write the following code. Keep in mind that Python is case-sensitive and that forgetting a letter or a required symbol will result in an error!

```
print("Hello world!")
```

Before executing the code, it is beneficial to clean your **History Pane** first to better see what will happen. After that, you can run your code by using either the **Execute All** or the **Execute** button. If you are using the second one, make sure to **highlight** your code first (select it) to prevent Maya from deleting it. The shortcut for "Execute" is **CTRL + Enter**.



(3.1.a Cleaning the History Pane, then running the code)

Note

By default, the Script Editor includes a MEL tab for writing code. Since this and the following examples are written in Python, click the **plus (+)** button in the **Input Pane** and add a Python tab.

When you run, you will get a result like this in the **History Pane**:

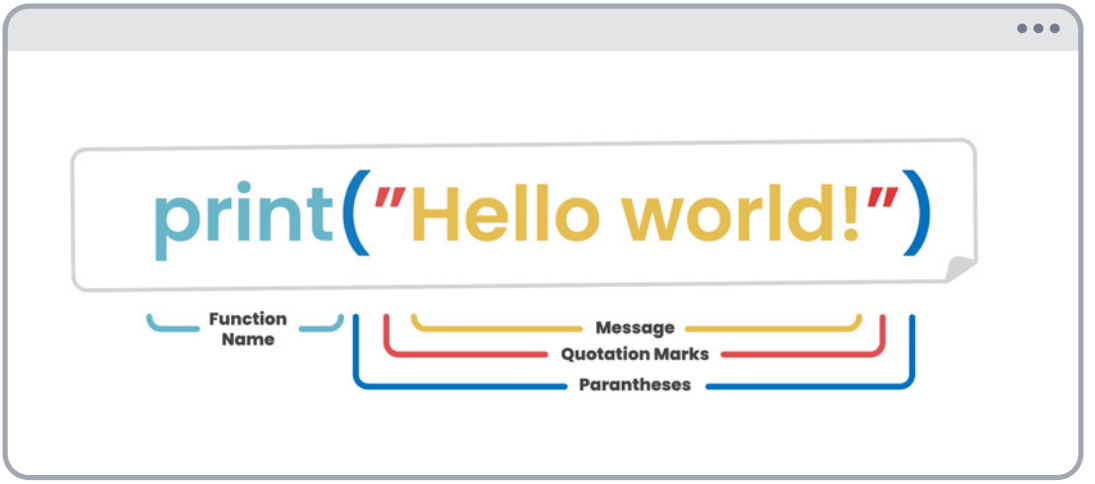
```
print("Hello world!")
Hello world!
```

The way Maya's Script Editor works is a little different from traditional IDEs. In Maya, the **History Pane** always displays the code that was executed first, followed by the **output** produced by that code.

```
print("Hello world!")   # This is the echo of the executed code.
Hello world!            # This is the output produced by the code.
```

So, when we run and test our scripts, the **output** is the essential part to focus on, as it is the actual result. This is easier with IDEs, as their console only prints the output.

Let's look into what is happening with this print statement in detail.



(3.1.b Print function body)

- When we type `print()` Python recognizes it.
- Parentheses ( ) are used after print to call the function.
- Quotation marks " " are used to show the start and end of the text message.
- Lastly, we type our **message** within the quotation marks.

Whatever we write within quotation marks is not code and can be replaced with any text. The quotation marks simply tell the computer that the text placed within them is not code. Otherwise, Python will always try to see written text as code and will display an error message if it does not recognize it.

You can try printing different messages, for example:

```
print("My name is x.")
print("This is a test!")
print("There are 6 meshes in the scene...")
```

Now, you might be thinking that this does not feel like coding because you are basically making the computer repeat whatever message you type. Still, print statements will be helpful and needed often in cases such as:

- To give feedback to users that the code was executed successfully, or if there was an issue.
- To test and debug your code.

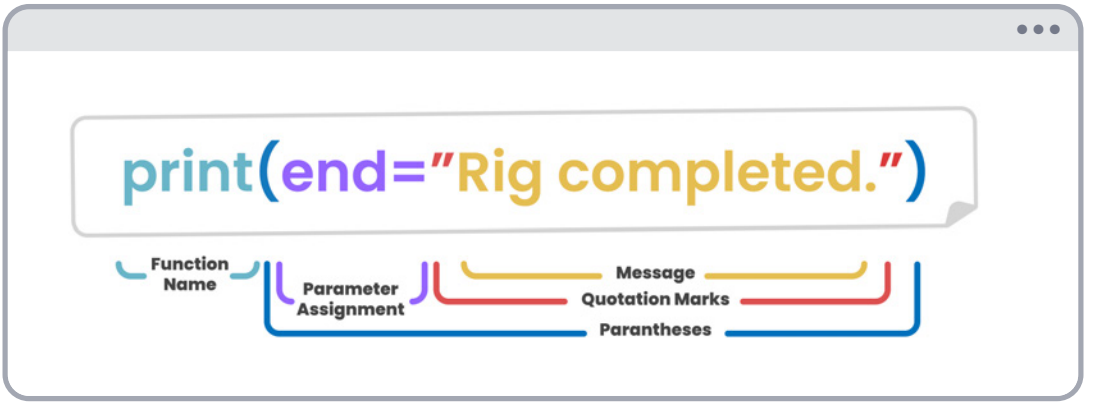
For example:

```
print("10 joints created.")
print("You picked an invalid option, returning to defaults.")
```

The `print()` function can be used with a parameter called `end` . This makes Maya print the output in the **Command Response** located at the bottom-left of the screen. This is useful for giving feedback.

Note

By default, the `print()` function adds a new line at the end of every message. The `end` parameter replaces that new line with the text you provide, so writing `print(end="your text")` with no main message means only that text is sent, which is what Maya shows in the Command Response.



(3.1.c Printing with `end`)

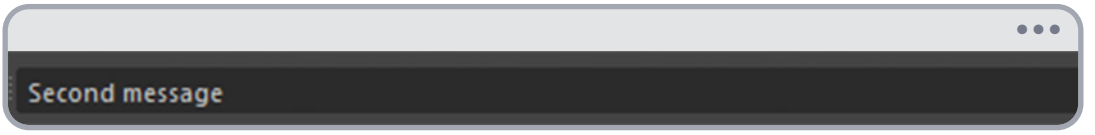
Try this with your own message. Type the following code in the Input Pane and execute.

```
print(end="This is my message.")
```

Note that if you use print statements with the `end` parameter multiple times in your code, Maya will show only the last one. For example, if you type the following code:

```
print(end="First message.")
print(end="Second message")
```

Maya will show only the second message in the **Command Response,** as shown in the following reference:



(3.1.d Message printed in Command Response)

3.2 Syntax

Syntax is a word that you will come across frequently while learning to code. It refers to the rules that define how code must be written for the program to understand it. For example, not following the syntax rules in a `print()` statement, such as using the wrong type of parentheses or forgetting to close quotation marks, can lead to errors. Below are some examples of incorrect syntax.

```
print("Message"          # Parentheses are not closed
print(Message")          # First quotation mark is missing
Print("Message")         # 'print' should be written in lowercase
print{"Message"}         # Parentheses should be used, not curly braces
```

Different Python concepts have different syntax rules, and as we progress, we will learn how to write them.

Comments.

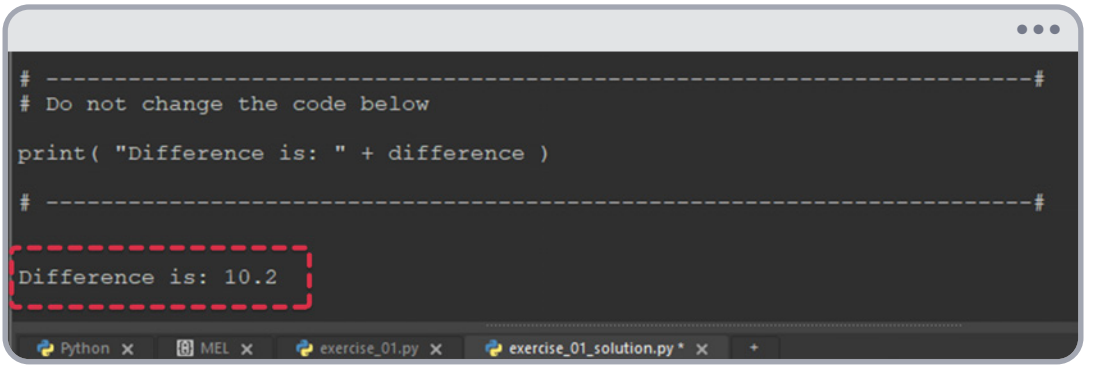
Comments are a helpful part of coding. In Python, comments start with the hash symbol `#`, and the program ignores anything written after it on the same line. Comments are used to explain code or add notes for readability for the benefit of others reading your code, or simply to remind yourself about what you have done. See the example below:

```
# This is a comment.
print("Example") # This prints a message.
```

There is something you need to consider when writing code in Maya's Script Editor as it has a special behaviour that sets it apart from traditional IDEs, and this can be a little confusing at first.

As covered in Section 3.1, the History Pane echoes the executed code before showing the output, which can make the actual result difficult to spot. Remember that the result is always at the very end, and it is what you should focus on.

For example, in Figure 3.2.a, the last line marked in color is the actual result. Anything that is printed above that is simply a copy of the code executed.



(3.2.a Actual result of the operation)

Quiz 3.2.1: Printing.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Print a message in the History Pane.

For example, you can write: `This is my message`.

Quiz 3.2.2: Printing in Command Response.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Print a message in the Command Response.

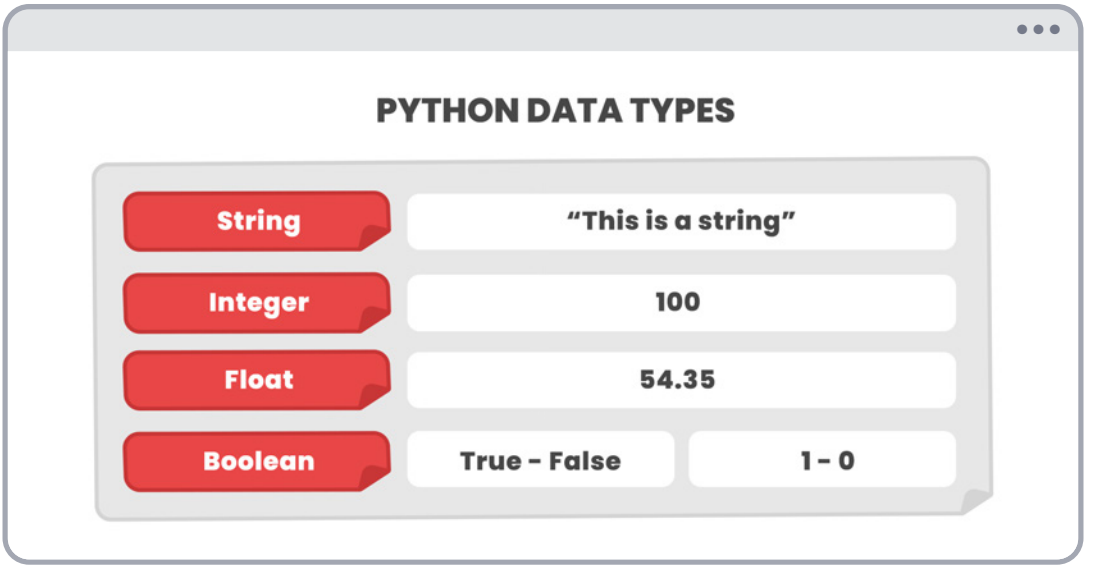
For example, you can write: `Code successfully executed`.

Note

Remember that the solutions can be found in the book's project folder. You can download it directly from your Jettelly account, Section: My Courses.

3.3 Data Types

In Python, data types define the kind of value a piece of data represents and determine the operations that can be performed on it. For example, if you want to perform some mathematical operations, the data type must be a numerical type such as an integer or a float. Figure 3.3.a shows the fundamental data types that are used most often.



(3.3.a Some Python Data Types)

String.

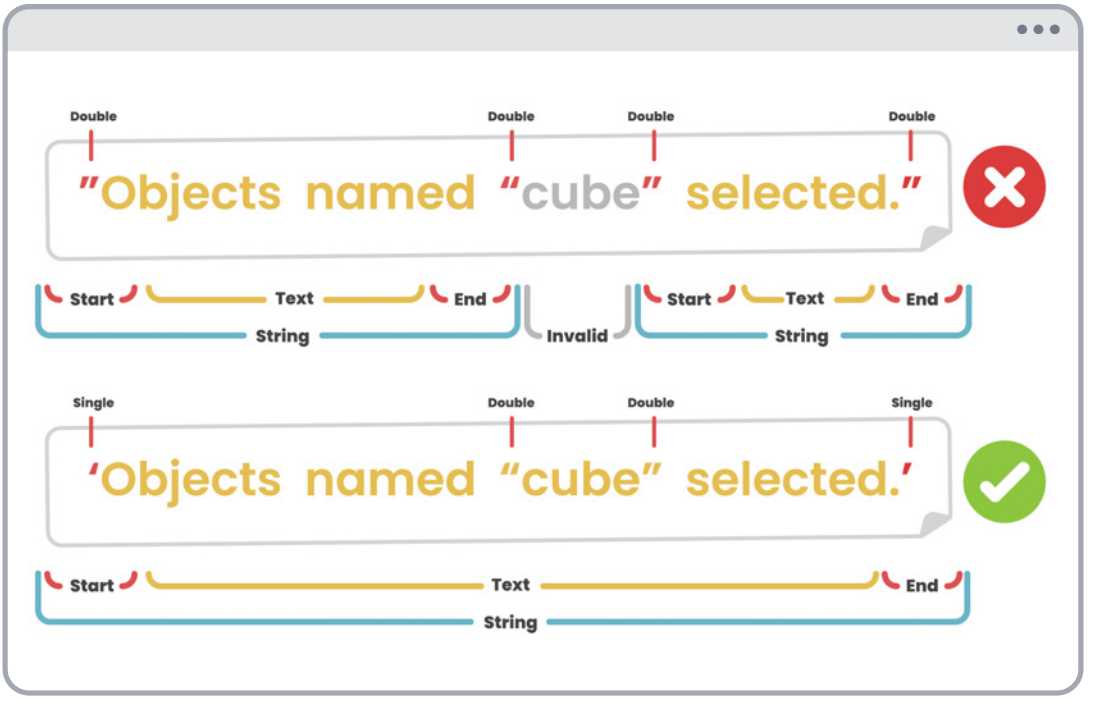
A string is a type of data used to represent common text. You might have recognized it from using it to print messages in the previous section. There are several ways to create strings:

- **Single Quotes** 'Text'
- **Double Quotes** "Text"
- **Triple Quotes** "' Three single quotes"' or """ Three double quotes"""

Single and **double quotes** are used to indicate the start and end of a string. When you need to use quotation marks **inside** your message, single and double quotes are used interchangeably to avoid confusion about where the string ends. Below are the basic rules:

- Single quotes can be used in double quotes.
- Double quotes can be used in single quotes.
- The string must start and end with the same type of quotation mark.

Let's try this with the following example message: Objects named "cube" selected. Because double quotes are used as a part of the text, this message should be placed within single quotes.



(3.3.b Quotation style)

Test out the `print()` statements provided below in the Script Editor one by one to see how they work and what kind of errors incorrect ones cause.

```
print('Objects named "cube" selected.')    # Double quotes inside single.
print("Objects named 'cube' selected.")    # Single quotes inside double.

print("Objects named "cube" selected.")    # Incorrect: double in double.
print('Objects named 'cube' selected.')    # Incorrect: single in single.

print("Text')        # Incorrect: Must end with the same type of quotes.
```

Triple quotes also mark the start and end of a string, but they allow us to write multi-line text, unlike single and double quotes. Try out the print statement below; you can replace the single triple quotes with double ones. Just remember the closing quotes must match the opening quotes!

```
print('''Message line 1...
Message line 2...
Message line 3...''')
```

Triple quotes are often used at the beginning of a code file to give information about the script, such as:

```
"""
MAYA RENAMER V3.0
Author: Zeynep Fisher
Date: Nov 2024
Contact: youremail@email.com
About: A Tool for renaming objects in Autodesk Maya.

Last Updated: May 2025
"""
```

Depending on where they are used, triple quote strings can be called **docstrings**. The example above is called a module docstring. Docstrings can also appear at the beginning of a function, class, or method to provide an explanation. We will discuss these in more detail in later sections as we learn about those concepts. For now, just note that if you are writing docstrings, the Python style guide recommends using triple double quotes.

Newline Escape Sequence (\n).

There are special backslash and letter combinations that tell Python to interpret certain parts of a string in a different way. These are called escape sequences, and `\n` is one of the most common ones.

This notation helps you insert a new line in a string. This way, you can print text in multiple lines without the need to use triple quotes, which in some cases might look messy. Try out the example below.

```
print("Line 1\nLine 2\nLine 3")

# Result:
# Line 1
# Line 2
# Line 3
```

Integer.
An integer (or int) is a data type in Python that represents whole numbers, such as 3, 50, -24, 0, or 1357. Integers are used for operations like counting, iterating, and indexing. In other programming languages, this data type has different precisions. However, in Python, the precision is limited only by the amount of available memory, not by a fixed number of bits, so the theoretical range is $-\infty < int < +\infty$.

Float.
Float (or floating-point) is a data type in Python that represents numbers with a decimal point, such as 37.45, -2.00, 65.12349, or 0.5. They are used for representing measurements, percentages, RGB color values, and calculations that involve fractions or decimals. Keep in mind that floats can carry tiny rounding errors, so a calculation such as 0.1 + 0.2 may not return exactly 0.3. This is normal in Python and in most programming languages.

Boolean.
Boolean (or bool) is a data type in Python that represents truth values. It can have only one of the two possible values: True or False, or, in numerical representation, 1 or 0. Booleans are often used in conditions, comparisons, and logical operations. For example, if we have a controller, we can determine whether it is selected by returning True or False.

Whenever you are unsure which type a value holds, you can confirm it with the `type()` function, which we cover in section 3.5.

Quiz 3.3.1: Printing in Multiple Lines.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Print a message that consists of three words.
3. Use `\n` to print each word in a separate line.

For example, you can write: `Mode, Rig, Animation`, or any other three words of your choice.

Quiz 3.3.2: Creating a Module Docstring.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Imagine you are creating your own Python tool and need to give a brief description of what it is. To do this, create a **module docstring** and include the following information:
   a. `Name of the tool`.
   b. `About`.
   c. `Author name`.
   d. `Date`.

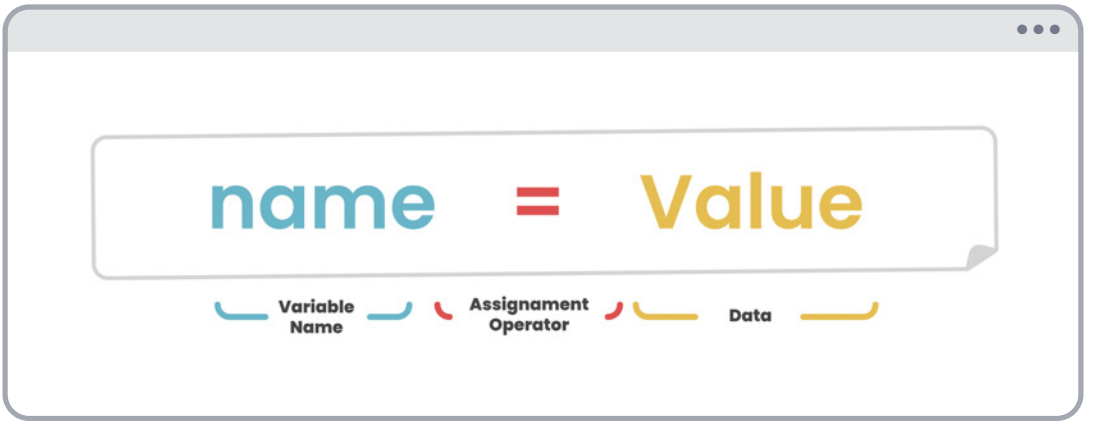
Quiz 3.3.3: Printing with Quotation Marks.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Print the following and include the quotation marks in the message:
   `No object named "character" found in the scene`.

3.4 Variables

A variable is a name or label that represents a piece of data in programming. It is like a container that holds a value (that varies over time), and can be accessed using its assigned name.



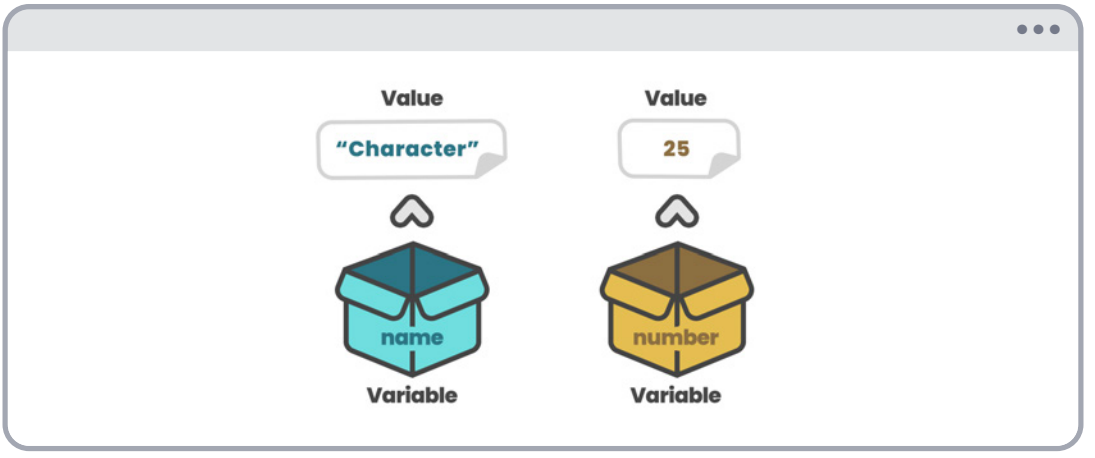
(3.4.a The body of a variable)

In the example below, each string is assigned to a variable (**name**, **prefix** and **suffix**). We then access each string using its assigned variable name and print a combined string. Try this in the Script Editor:

```
name = "Eye"
prefix = "R_"
suffix = "_GEO"

print(prefix + name + suffix)

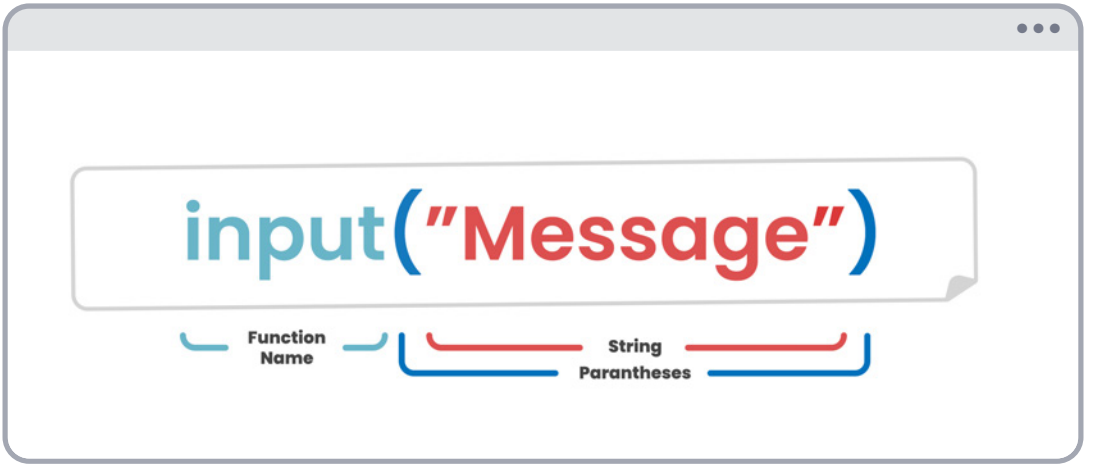
# Output: R_Eye_GEO
```



(3.4.b A string, and an integer variables)

Let's try the same code with user input this time. Python has an `input()` function that allows you to collect data from the user through an input window.

It is a built-in function like `print()`, and such functions usually follow similar syntax: the function name followed by parentheses. You can include a message to guide users about what type of data you are looking for in the `input()` function.

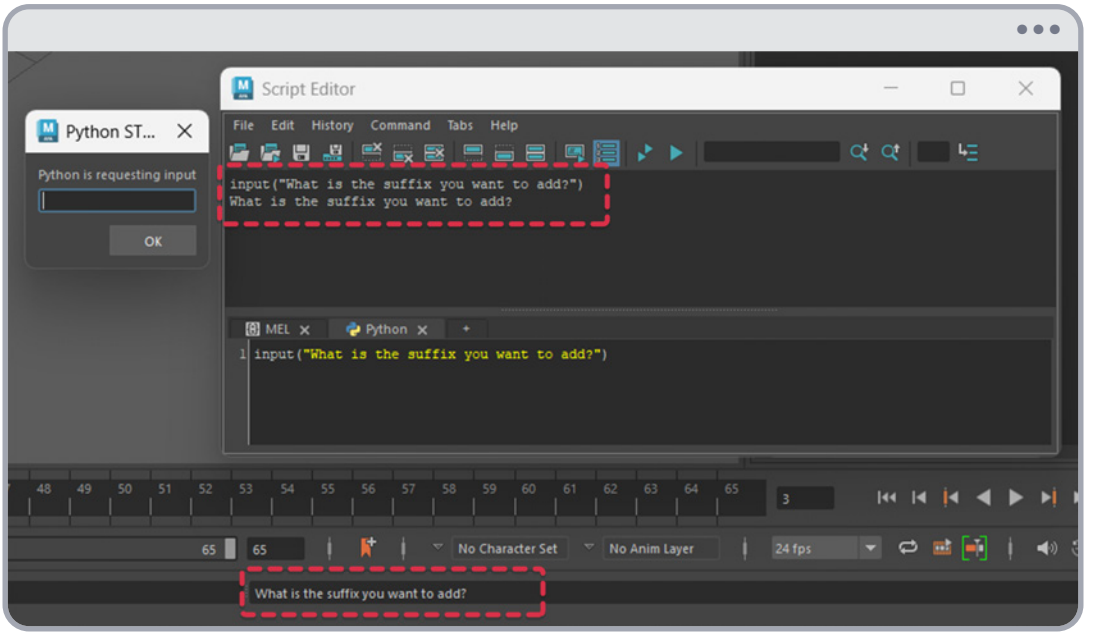


(3.4.c Input function body)

Run the code below in your Script Editor. You can replace the message with your own:

```
input("What is the suffix you want to add?")
```

On the line of the `input()` function, Maya will pause the execution and open a window, prompting you to enter an input. Both the **History Pane** and the **Command Response** will display the relevant message for that input window, as shown in Figure 3.4.d. The **Command Response** may be easier to read since it only displays the input message, whereas the **History Pane** first shows you the echo of the code. However, if your input message includes multiple lines of text, the **History Pane** will be easier to read, because the **Command Response** will condense your text into a single line.



(3.4.d Maya is requesting an input)

Once you enter your input, the code will continue executing. Here, the question is: how can we access this input later if we want to use it inside our code? The answer is that we simply need to store it in a variable.

Let's try the code below. The output should be the combination of your three inputs. Add `\n` to insert new lines after each message to improve readability in the History Pane.

Also, an important note: many examples in this book use the `print()` function frequently. This is because when the computer performs an action internally, we need a way to make the result visible to us. One of the simplest ways to do this is by printing to the screen. For example, if we store user input in variables, printing them lets us check whether the data was saved correctly, whether the result is accurate, and whether the code is working as expected (no errors found). Without output, we receive no clear feedback from the computer. Technically speaking, this process is called "debugging."

```
name = input("Type a name\n")
prefix = input("Type a prefix\n")
suffix = input("Type a suffix\n")

print(prefix + name + suffix)
```

To further clarify what is happening in the code above, we use the `input()` function to receive data from the user and store it in a variable. This process is repeated three times, saving data in the variables `name`, `prefix`, and `suffix`. After that, we use the `print()` function to display the combined result of these variables.

The important point is that, without storing the input in variables, the data vanishes, and there is no way to reference it later in the code. This is why variables are crucial in coding.

Let's continue with the variables. You can assign different data types to variables. Test out the code blocks below. You will notice that because the variables are assigned to integer or float data types, using the plus or minus symbol will work as a mathematical operation, unlike combining strings.

```
model_count = 10
curve_count =  5

print(model_count + curve_count) # Result: 15
```

Or,

```
cube_x_pos = 10.5
sphere_x_pos = 22.5
distance = sphere_x_pos - cube_x_pos

print(distance) # Result: 12.0
```

Another helpful feature of Python is that variables can be reassigned and are dynamically typed.

Reassigning means that the value stored in a variable can be changed or updated during program execution. For example, if you run the code below, it will print 10 because that is the latest assigned value of the name variable, as the code is always executed from top to bottom.

Dynamic typing means that a variable is not fixed to a single data type, and its type can change depending on the value it stores. This is different from statically typed languages such as C++, where data types must be explicitly declared. This makes Python flexible and faster to write. In the example below, the name variable initially has a string type (`"Cat"`), which is later changed to an integer (10).

```
name = "Cat"
name = 10

print(name) # Result: 10
```

Variable Naming Rules.

We can give any name to our variables as long as they are consistent, but still, there are a few rules that we should keep in mind.

1. Use clear, meaningful variable names for readability. For example, if you name a variable "`m`", you may not remember what it refers to the next time you open your code. Instead, an understandable name would be helpful, such as `model`, `mesh_name` or `material_type`.
2. Keep variable names sensible in length. Python does not limit the length of a name, but for readability, it is best to make names descriptive and understandable without being so long that they are hard to read, type, or maintain. For example:
   a. Long: `number_of_polygons_selected_in_the_scene.`
   b. Better alternative: `visible_poly_count`
3. To prevent errors, avoid using names of Python's built-in functions or keywords, such as **print** or **input**. The same applies to Maya's command names, which we will discuss in the fourth chapter.
4. Variable names must not start with a number. For example, `12th_player` is an invalid name.
5. Variable names cannot contain spaces. For multiple words, use underscores to separate, such as `my_variable`.
6. Apply "Snake Case," which is the recommended style. It uses lowercase words separated by underscores, e.g. `my_variable`. The following styles are valid in Python but are not considered good practice for variable names.
   a. Pascal Case: `MyVariable`
   b. Camel Case: `myVariable`
   c. Mixed Case: `My_Variable - My_variable - my_Variable`

Quiz 3.4.1: Variables I.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Create a variable called `name`.
3. Assign a text value to the `name` variable.
4. Print the message: Hello `name`!
5. The code should replace name with the value the variable holds when printed.

For example:

- You can assign `"cat"` to the variable. The code should print: `Hello Cat!`

Quiz 3.4.2: Variables II.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Create a variable called `file_name` and assign a value.
3. Create another variable called `extension` and assign a value.
4. Combine `file_name` and `extension`, and print the result.

Example:

- `file_name` can be `BaseColor`.
- The extension variable can be `.png`.
- The code should output: `BaseColor.png`

Quiz 3.4.3: Variables III.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Create a variable called `prop_count` and assign an integer value.
3. Create another variable called `character_count` and assign an `integer` value.
4. Print the sum of the `prop_count` and `character_count` variables.

Example:

- `prop_count` can be 10.
- `character_count` can be 25.
- The code should output: 35.

Type Casting.

As you progress in coding, you will notice that using the correct data type and regularly checking the type will become increasingly crucial for the program to run successfully. Try the following code and enter numbers such as 8 and 15. Try to guess the result. Is it going to print the sum of the numbers, or something else?

```
cube_count = input("how many cubes would you like to create?\n")
sphere_count = input("how many spheres would you like to create?\n")

total_num = cube_count + sphere_count
print(total_num)
```

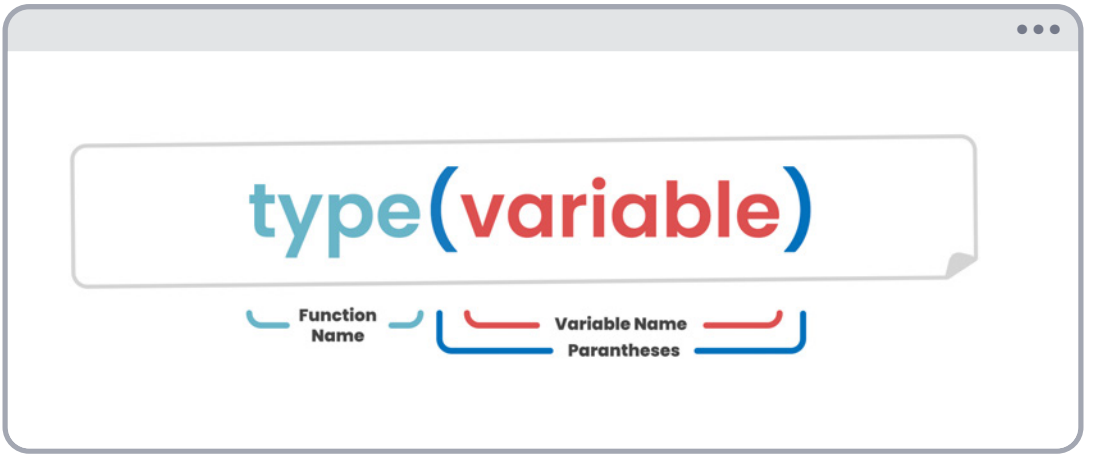
The output will show the numbers placed side by side rather than adding them together mathematically. For example, if you typed 10 and 25, it will show you 1025. This is because the `input()` function always returns a string (a text), even if the input is a number. You can check the data type by using a function called `type()`, as shown in the code below.

```
cube_count = input("how many cubes would you like to create?\n")

print(type(cube_count)) # Get the data type and print it.

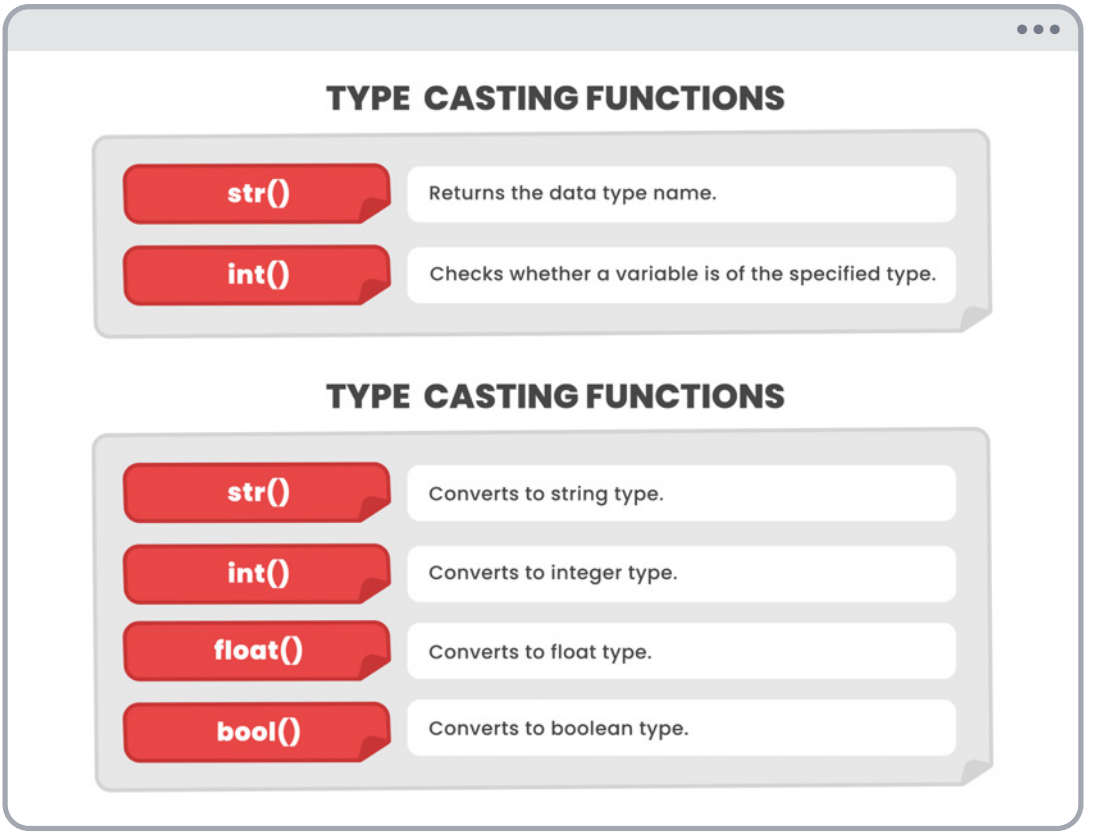
# result: <class 'str'> (str means string)
```

The `type()` function returns the name of the data type of the variable specified in the parentheses. If you print it, it typically looks like `<class 'str'>`. The part that comes after the word "class" specifies the data type, so you might get other results such as `'float'`, `'int'`, `'bool'`, or other types that we have not discussed yet.



(3.4.e Body of the `type()` function)

Let's attempt the previous code exercise again. To convert strings to numbers so that we can successfully apply math operations, we can use one of the type-casting functions, as shown in following reference.



(3.4.f Type casting functions)

As we are asking for the number of cube and sphere objects to be created, it makes sense to turn them into integers rather than floats. Try out the code below. Keep in mind that if the inputs contain words such as "three" or "four" instead of numbers, the `int()` function will raise an error because only numerical characters can be converted to integers or floats. For the same reason, a decimal string such as `int("3.5")` also raises an error, while converting a float such as `int(3.9)` returns 3, since `int()` drops the decimal part rather than rounding it.

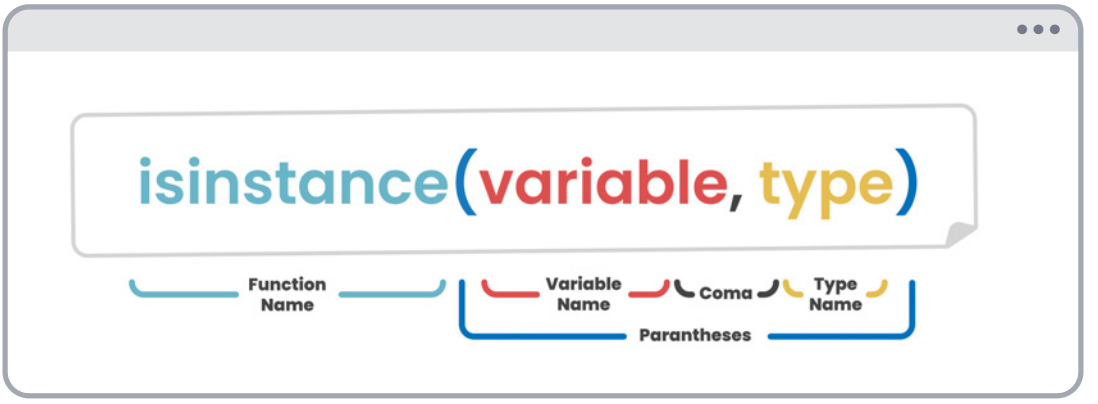
```
cube_count = input("how many cubes would you like to create?\n")
sphere_count = input("how many spheres would you like to create?\n")

total_num = int(cube_count) + int(sphere_count) # Type Casting
print(total_num)
```

This code should now return 10 + 25 = 35 (rather than = 1025).

Type casting also works in the other direction. When you need to place a number inside a text message, you can convert it to a string with the `str()` function, for example `str(15)`.

Another way to check the type of a variable is the `isinstance()` function. This function checks whether a variable belongs to a specific data type. You provide the variable you want to check and the data type you want to compare it to. If the variable matches the specified data type, the function returns **True**; otherwise, it returns **False**.



(3.4.g The body of the `isinstance()` function)

Try out the code below and compare the variables with data types:

```
variable1 = 10
variable2 = 35.6
variable3 = "There are 5 objects."
variable4 = True

# Statements below will print True.
print(isinstance(variable1, int))
print(isinstance(variable2, float))
print(isinstance(variable3, str))
print(isinstance(variable4, bool))

# The code continues on the next page.
```

```
# Statements below will print False.
print(isinstance(variable1, float))     # variable1 is int, not float.
print(isinstance(variable2, int))       # variable2 is float, not int.
print(isinstance(variable3, bool))      # variable3 is str, not bool.
print(isinstance(variable4, str))       # variable4 is bool, not str.
```

Quiz 3.5.1: Input.

Instructions:

1. Add a new Python coding tab in the Script Editor.
2. Get user input with the following message: `Type a color name`, and store it in a variable.
3. Print the input in a sentence.

Example:

- You could write: `The default color is changed to yellow`.
- `yellow` should be replaced with the user input.

Quiz 3.5.2: Type Casting.

Instructions:

1. Add a new Python tab in the Script Editor.
2. Create a variable called `height`, and assign a numeric value (integer or float), such as 15 or 8.5.
3. Turn `height` into a string.
4. Print the following message incorporating the height variable: `Object height is 15 cm`. The number in the sentence should change based on the height value.



Learn. Create. Master.